МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ И ПРОГРАММНОЙ ИНЖЕНЕРИИ

Задание
на курсовой проект по дисциплине
«Объектно-ориентированное программирование»

Студенту группы 4333К _____
№ группы Ф.И.О.

Тема «Разработка приложения для организации взаимодействия объектов при заданных критериях»

Исходные данные: Компьютерная ремонтная мастерская

Проект должен содержать:

- анализ предметной области
- разработку классов
- разработку тестового приложения
- оформление пояснительной записки по результатам выполнения проекта
- создание презентации к проекту

Срок сдачи законченного проекта 31.12.2025

Руководитель проекта Ст.Пред Шумова Е.О.

Дата выдачи задания 02.09.2025

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 4

1. Постановка задачи 5

1.1. Анализ предметной области 5

1.2. Формулировка функционала 5

2. Разработка классов 7

2.1. Классы сущностей 7

2.2. Управляющие классы 12

2.3. Интерфейсные классы 13

2.4. Используемые паттерны проектирования 13

2.5. Диаграмма классов 16

3. Разработка приложения 18

3.1 Разработка интерфейса приложения 18

3.2 Реализация методов классов 25

3.3 Разработка тестового приложения 29

4. Тестирование 31

ЗАКЛЮЧЕНИЕ 42

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 43

ПРИЛОЖЕНИЕ 1. Исходный код 44

ВВЕДЕНИЕ

Компьютерная техника активно используется в работе, обучении и быту, поэтому тема качественного и своевременного ремонта остаётся актуальной. Для мастерских важно не только выполнять ремонт, но и эффективно организовывать взаимодействие между клиентами, мастерами и филиалами. Без автоматизации такие процессы становятся менее удобными, а без цифровизации значительно сложнее наладить взаимодействие между всеми процессами мастерской, что приводит к задержкам, ошибкам и снижению качества обслуживания.

Целью курсового проекта является разработка программного приложения для автоматизации работы компьютерной ремонтной мастерской. Система должна обеспечивать регистрацию клиентов и их устройств, формирование и сопровождение заказов, назначение мастеров с учётом их квалификации, а также ведение данных о филиалах.

Для достижения цели в основной части работы был проведён анализ предметной области, выделены ключевые сущности: клиент, мастер, филиал, заказ, объект ремонта. Была разработана структура классов в соответствии с требованиями предметной области, выявленными после анализа, спроектирован и разработан интерфейс приложения и реализована его функциональность.

1. Постановка задачи

1.1. Анализ предметной области

В качестве предметной области была выбрана “Компьютерная мастерская”, данная область представляет из себя взаимодействие множества сущностей, но ключевыми были выбраны:

- Клиент – сущность, отражающая клиента, который принёс в мастерскую определённый объект, который необходимо починить
- Мастер – сущность, которая является работником в мастерской, который занимается починкой объектов на определённой точке ремонта, среди мастеров выделяется различные уровни квалификации.
- Объект для починки – сущность, представляющая из себя объект, который принёс клиент в мастерскую для последующей починки
- Заказ – центральная связующая сущность, которая объединяет между собой клиента, мастера и объект заказа для отражения статуса ремонта.
- Точка ремонта – сущность, отражающая в себе филиал мастерской, в которой и происходит основной цикл ремонта, каждая точка имеет за собой закреплённых мастеров.

Каждый заказ формируется работником, принимающим клиента, за заказом закрепляется клиент, мастер и, соответственно, объект для ремонта. Один мастер может работать только над одним заказом, когда как клиент может сдать несколько объектов в ремонт. За каждым мастером закреплён филиал, Мастер не может работать с объектом ремонта, который закреплён на другой филиал.

1.2. Формулировка функционала

На основе анализа предметов области были выявлены основные требования к функциональности системы:

- Добавление клиента – возможность зарегистрировать клиента в системе с добавлением его персональной информацией для будущих контактов
- Добавление мастера – возможность зарегистрировать нового мастера на определённую точку с определённой должностью (Junior, Middle, Senior, Other)
- Увольнение мастера – деактивация мастера, не позволяет больше приписать заказы данному мастеру
- Добавление объектов – регистрация объектов для ремонта, которые приносит клиент, с привязанностью к филиалу.

- Добавление филиала – создание нового филиала мастерской с названием, адресом и номером телефона
- Возобновить/приостановить работу филиала – Отключение возможности добавления заказа для данного филиала на неопределённый срок, а также включение этой возможности обратно.
- Добавление заказа – Создание нового заказа с привязкой к клиенту, мастеру и объектом для ремонта. Невозможно добавить заказ мастеру с уже существующим заказом и филиалу с отключенной возможностью принимать заказы
- Выполнение/Отмена заказа – Возможность выполнить или отменить заказ, а также освобождение мастера от заказа вследствие этого действия.
- Просмотр истории – Возможность просмотреть историю каждого действия, совершённого в системе.

2. Разработка классов

2.1. Классы сущностей

Класс Client является отражением сущности Клиент, которая была выделена на постановке задачи, и отражает физическое или юридическое лицо, которое пользуется услугами ремонтной мастерской. Далее представлен хедер класса сущности Клиент:

```
class Client : public QObject{Q_OBJECT
public:
   explicit Client(QObject *parent = nullptr, QString FIO = "", QString Adress="", QString PhoneNumber="");
   static int PhoneNumberToRegNumber(QString);
   int GetRegNumber();
   QString GetFIO();
   QString GetAdress();
   QString GetPhoneNumber();
private:
   int RegNumber;
   QString FIO;
   QString Adress;
   QString PhoneNumber;};
```

Данная сущность содержит:

- Поля:
  - RegNumber – Регистрационный номер клиента
  - FIO – ФИО клиента
  - Adress – Адрес клиента
  - PhoneNumber – Номер телефона клиента
- Геттеры:
  - GetRegNumber – Возвращает номер клиента
  - GetFIO – Возвращает ФИО клиента
  - GetAdress – Возвращает адресс клиента
  - GetPhoneNumber – Возвращает номер телефона клиента
- Конструктор класса с назначением указанных выше полей

Класс Master является отражением сущности Мастер, которая была выделена на постановке задачи, и работника мастерской определённой квалификации, который занимается исполнением заказов. Далее представлен хедер класса сущности Мастер:

```
class Master : public QObject {Q_OBJECT
public:
    enum class MasterStatus{
        InWork,
        Fired};
    explicit Master(QObject *parent = nullptr, QString FIO="", QString Position="", QString Place="");
    static QString FIOnPosToRegCode(QString, QString);
    void SetOrder(int);
    void SetStatus(bool);
    QString GetRegCode();
    QString GetFIO();
    QString GetPosition();
    QString GetPlaceName();
    MasterStatus GetStatus();
    QString GetStatusString();
    bool IsBusy();
    QString GetOrderStatus();
private:
    QString RegCode;
    QString FIO;
    QString Position;
    QString Place;
    MasterStatus Status;
    bool MasterBusy;
    int OrderNumber;};
```

Данная сущность содержит:

- Поля:
  - RegNumber – Регистрационный номер мастера
  - FIO – ФИО мастера
  - Position – Должность мастера
  - Place – Филиал мастера
  - Status – Статус работы мастера (Работает/Уволен)
  - MasterBusy – Занят ли мастер заказом
  - OrderNumber – Номер заказа (если нет то -1)

- Геттеры
    - GetRegCode – Возвращает регистрационный номер мастера
    - GetFIO – Возвращает ФИО мастера
    - GetPosition – Возвращает должность мастера
    - GetPlaceName – Возвращает филиал мастера
    - GetStatus – Возвращает статус работы мастера (Работает/Уволен)
    - GetStatusString – Возвращает статус работы в виде строки
- Сеттер для заказа и статуса работы
- Конструктор класса с назначением полей

Класс Thing является отражением сущности Объект ремонта и представляет себя объект, который Клиент сдаёт в Филиал мастерской для починки. Далее представлен хэдер сущности Объект ремонта:

```
class Thing : public QObject {Q_OBJECT
public:
   enum class ThingStatus{
      InWork,
      None};
   explicit Thing(QObject *parent = nullptr, QString ThingName="", QString PlaceName="");
   QString GetThingName();
   QString GetPlaceName();
   QString GetThingStatusString();
   bool IsThingInOrder();
   void SetOrder(int);
private:
   QString ThingName;
   QString PlaceName;
   ThingStatus Status;
   bool ThingInOrder;
   int OrderNumber; };
```

Данная сущность содержит:

- Поля:
    - ThingName – Название объекта
    - PlaceName – Филиал нахождения
    - Status – Статус объекта (В работе/Нет)

- ThingInOrder – В заказе ли объект
- OrderNumber – Номер заказа объекта (если нет то -1)

- Геттер
  - GetThingName() – Возвращает имя объекта
  - GetPlaceName() – Возвращает имя филиала
  - GetThingStatusString() – Возвращает статус(В работе или нет) в виде строки
- Сеттер для заказа
- Конструктор класса с назначением полей

Класс Place представляет из себя сущность Филиал, которая отражает отделения ремонтной мастерской. Далее представлен хедер класса сущности Филиал:

```
class Place : public QObject { Q_OBJECT
public:
   enum class PlaceStatus{
      InWork,
      Suspended };
   explicit Place(QObject *parent = nullptr, QString PlaceName = "", QString PlaceAdress = "", QString PhoneNumber = "", PlaceStatus Status = PlaceStatus::InWork);
   QString GetPlaceName();
   QString GetPlaceAdress();
   QString GetPhoneNumber();
   PlaceStatus GetPlaceStatus();
   QString GetStringPlaceStatus();
   void EnablePlace();
   void DisablePlace();
private:
   QString PlaceName;
   QString PlaceAdress;
   QString PhoneNumber;
   PlaceStatus Status; };
```

Данная сущность содержит:

- Поля:
  - PlaceName – Название филиала
  - PlaceAdress – Адрес филиала

    - PhoneNumber – Номер телефона филиала
    - Status – Статус филиала (Работает/Нет)
- Геттеры
    - GetPlaceName() – Возвращает имя филиала
    - GetPlaceAdress() – Возвращает адрес филиала
    - GetPhoneNumber() – Возвращает номер телефона
    - GetPlaceStatus() – Возвращает статус филиала
    - GetStringPlaceStatus() – Возвращает статус филиала в виде строки

- Конструктор класса с назначением полей
- Методы для изменения статуса филиала

Класс Order представляет из себя сущность Заказ, которая является связующим звеном системы между остальными сущностями. Она отражает заказ, который составляется, когда Клиент отдаёт Объект на ремонт в Филиал, где Мастер исполняет процесс ремонта. Далее представлен хэдер класса сущности Заказ:

```
class Order : public QObject { Q_OBJECT
public:
   enum class RepairStatus{
      InProcess,
      Cancelled,
      Completed};
   explicit Order(QObject *parent = nullptr, int ClientRegNumber = 0,
            QString MasterRegCode = "", QString ThingToRepair = "");
   void SetOrderNumber(int);
   void SetRepairStatus(RepairStatus);
   int GetClientRegNumber();
   QString GetMasterRegCode();
   QString GetThingToRepair();
   RepairStatus GetStatusOfRepair();
   QString GetStringStatusOfRepair();
private:
   int ClientRegNumber;
   QString MasterRegCode;
   QString ThingToRepair;
```

RepairStatus StatusOfRepair;};

Данная сущность содержит:

- Поля:
  - ClientRegNumber – Регистрационный номер клиента
  - MasterRegCode – Регистрационный номер мастера
  - ThingToRepair – Объект починки
  - StatusOfRepair – Статус заказа (Выполнен/В работе/Отменён)
- Геттеры
  - GetClientRegNumber() – Возвращает регистрационный номер клиента
  - GetMasterRegCode() – Возвращает регистрационный номер мастера
  - GetThingToRepair() – Возвращает объект для ремонта
  - GetStatusOfRepair() – Возвращает статус ремонта
  - GetStringStatusOfRepair() – Возвращает статус ремонта в виде строки

- Конструктор класса с назначением полей
- Сеттер для статуса заказа

2.2. Управляющие классы

Для процесса управления взаимодействия между сущностями используются два управляющий класса, предназначенный для всех взаимодействий нескольких сущностей и окон интерфейса

1. MainWindow – Главный контроллер приложения

   Роль: Координатор сигналов внутри приложения

   Функци:

   - Открытие окон интерфейса
   - Передача сигналов из одного окна в другое паттерном “Наблюдатель”

   Методы класса:

   - Connect – реализация Наблюдателя для передачи сигналов
2. OrderManupilate – Фасад для решения взаимодействия нескольких сущностей

   Роль: Общий интерфейс для общения сущностей.

   Функции:

   - Добавить заказ
   - Выполнить заказ
   - Отменить заказ
   - Удалить мастера

   Методы фасада:

- AddOrder() – Добавление заказа
- ExecuteOrder() – Выполнение заказа
- RemoveOrder() – Отменить заказ
- RemoveMaster() – Уволить мастера

2.3. Интерфейсные классы

В приложении реализовано 6 интерфейсных классов, благодаря которым происходит взаимодействие с пользователем:

1. HistoryMenu – Окно для просмотра истории действий

   Роль:

   - Просмотр истории действий пользователей в системе
2. ClientsMenu – Окно взаимодействия с сущностью Клиент.

   Роль:

   - Добавление клиентов в систему
3. MastersMenu – Окно взаимодействия с сущностью Мастер.

   Роль:

   - Добавление мастеров в систему
   - Увольнение мастеров
4. OrdersMenu– Окно взаимодействия с сущностью Заказ.

   Роль:

   - Создание заказа
   - Завершение или отмена заказа
5. PlacesMenu– Окно взаимодействия с сущностью Филиал.

   Роль:

   - Добавление филиала
   - Изменение статуса филиала
6. ThingsMenu– Окно взаимодействия с сущностью Объект ремонта.

   Роль:

   - Добавление нового объекта для филиала

2.4. Используемые паттерны проектирования

В данной курсовой работе реализованы два паттерна проектирования: поведенческий “Наблюдатель” и структурный “Фасад”.

Паттерн “Наблюдатель” необходим для взаимодействия посредством сигналов между интерфейсными классами и основным классом MainWindow, где происходит вызов функций

для разрешения полученного запроса.

На рисунках 1-4 представлены диаграммы паттерна “Наблюдатель” с разными сигналами.

На рисунке 1 представлен сигнал при нажатии кнопки увольнения в интерфейсе взаимодействия с сущностью Мастер.

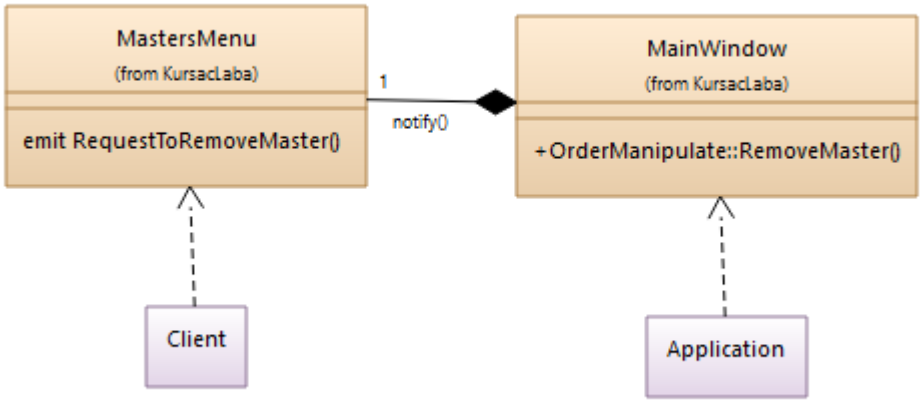


Рисунок 1 – Сигнал увольнения мастера

На рисунке 2 представлен сигнал добавления заказа при нажатии на кнопку добавления заказа в интерфейсе взаимодействия с сущностью Заказ.

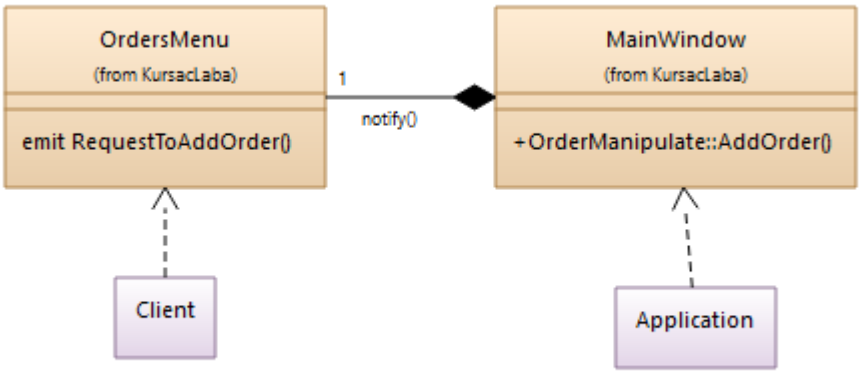


Рисунок 2 – Сигнал добавления заказа

На рисунке 3 представлен сигнал отмены заказа при нажатии на кнопку отмены заказа в интерфейсе взаимодействия с сущностью Заказ.

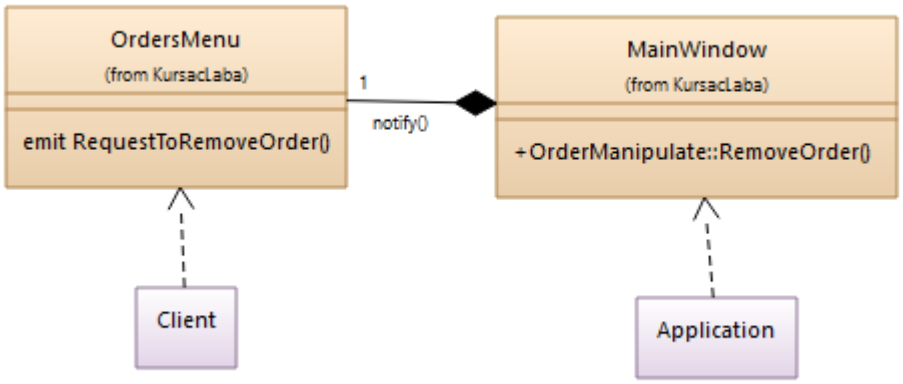


Рисунок 3 – Сигнал отмены заказа

На рисунке 4 представлен сигнал выполнения заказа при нажатии на кнопку выполнения заказа в интерфейсе взаимодействия с сущностью Заказ.

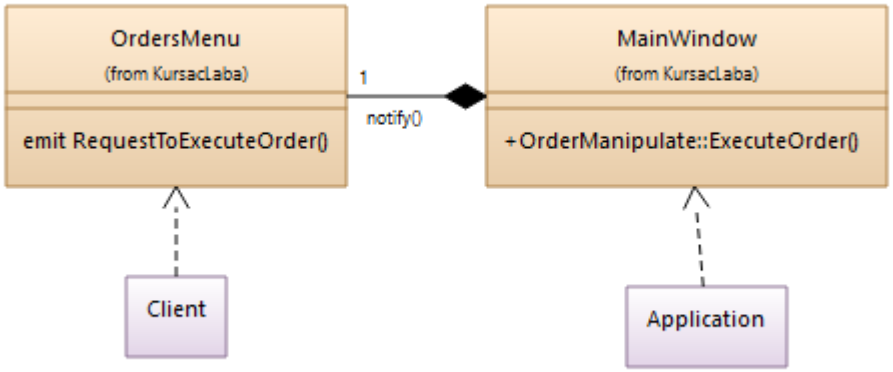


Рисунок 4 – Сигнал выполнения заказа

Паттерн “Фасад” необходим для реализации управляющего класса OrderManipulate, в основном классе MainWindow вызываются статические методы данного класса, которые впоследствии выполняет обработку запроса между несколькими сущностями. Полный функционал данного паттерна был описан в пункте 2.3 данной работы.

На рисунке 5 представлена схема паттерна “Фасад”

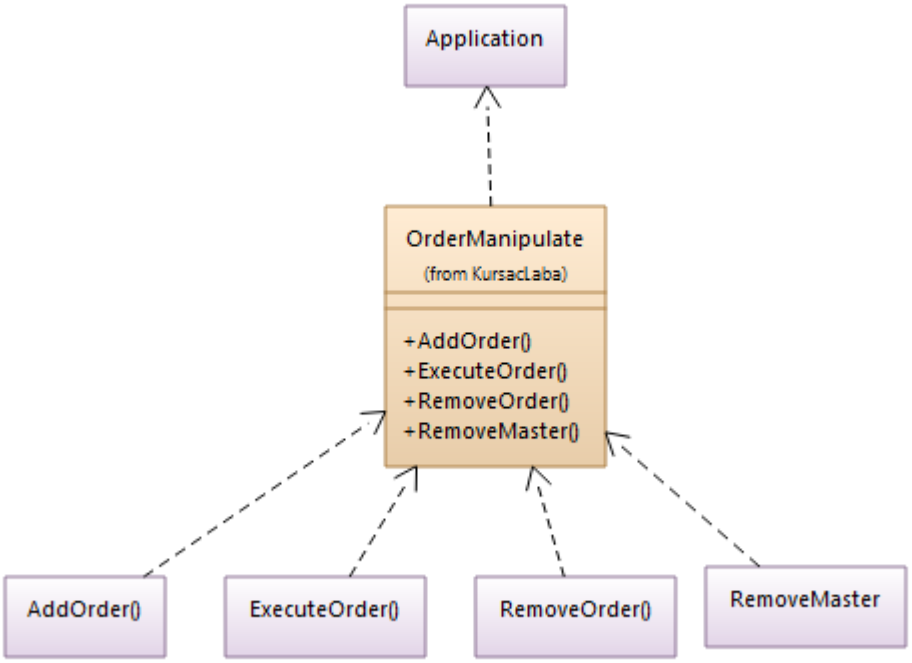


Рисунок 5 – Паттерн “Фасад”

2.5. Диаграмма классов

На рисунке 6 представлена общая диаграмма классов приложения.

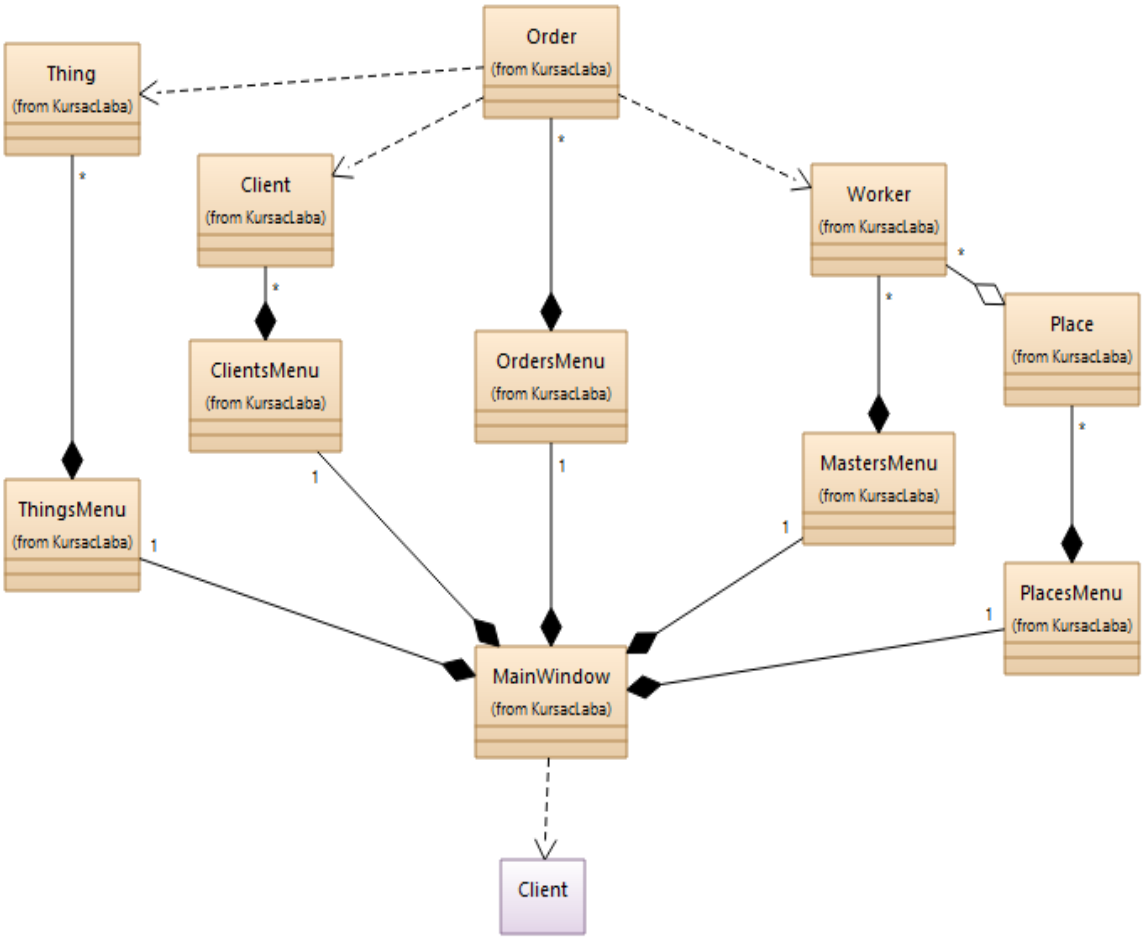


Рисунок 6 – Общая диаграмма классов

Приложение состоит из основного управляющего класса MainWindow, которое содержит 5 классов интерфейса, каждый из которых привязан к своей сущности. В каждом интерфейсе для сущности предусматривается вывод всех существующих сущностей, а также функции предусмотренные в постановке задачи.

Каждая функция так или иначе взаимодействует с сущностью, но если данное взаимодействие зависит от других сущностей, то отправляется соответствующий сигнал в MainWindow, где посредством фасада решается данная задача с учётом всех зависимостей.

3. Разработка приложения

3.1 Разработка интерфейса приложения

Разработка интерфейса приложения проводилась в среде QT Creator с помощью встроенного редактора. Для приложения были разработаны следующие формы интерфейса:

- Mainwindow – Основное окно при запуске приложения
- Clientsmenu – Меню взаимодействий с клиентами
- Mastersmenu – Меню взаимодействий с мастерами
- Ordersmenu – Меню взаимодействий с заказами
- Placesmenu – Меню взаимодействий с филиалами
- Thingsmenu – Меню взаимодействий с объектами ремонта
- HistoryMenu – Меню взаимодействий с историей действий

Далее будет рассмотрена каждая форма по отдельности с точки зрения их составляющих и программируемого функционала.

Mainwindow – Основное меню, которое появляется при запуске приложения, через него происходит основное взаимодействие с остальным функционалом приложения. Форма состоит из следующих элементов:

- SubTitle – Qlabel – Поле со значением текущей версии
- MainTitle – Qlabel – Поле с названием приложения
- HistoryButton – QpushButton – Кнопка для открытия HistoryMenu
- AllButtons – QVBoxLayout – Компоновка кнопок по вертикали
    - MainButtons – QHBoxLayout – Компоновка кнопок по горизонтали
        - ClientsButton – QpushButton – Кнопка открытия Clientsmenu
        - MastersButton – QpushButton – Кнопка открытия Mastersmenu
        - OrdersButton – QpushButton – Кнопка открытия Ordersmenu
    - MainButtons2 – QHBoxLayout – Компоновка кнопок по горизонтали
        - PlacesButton – QpushButton – Кнопка открытия Placesmenu
        - ThingsButton – QpushButton – Кнопка открытия Thingsmenu

Форма представлена на рисунке 7.

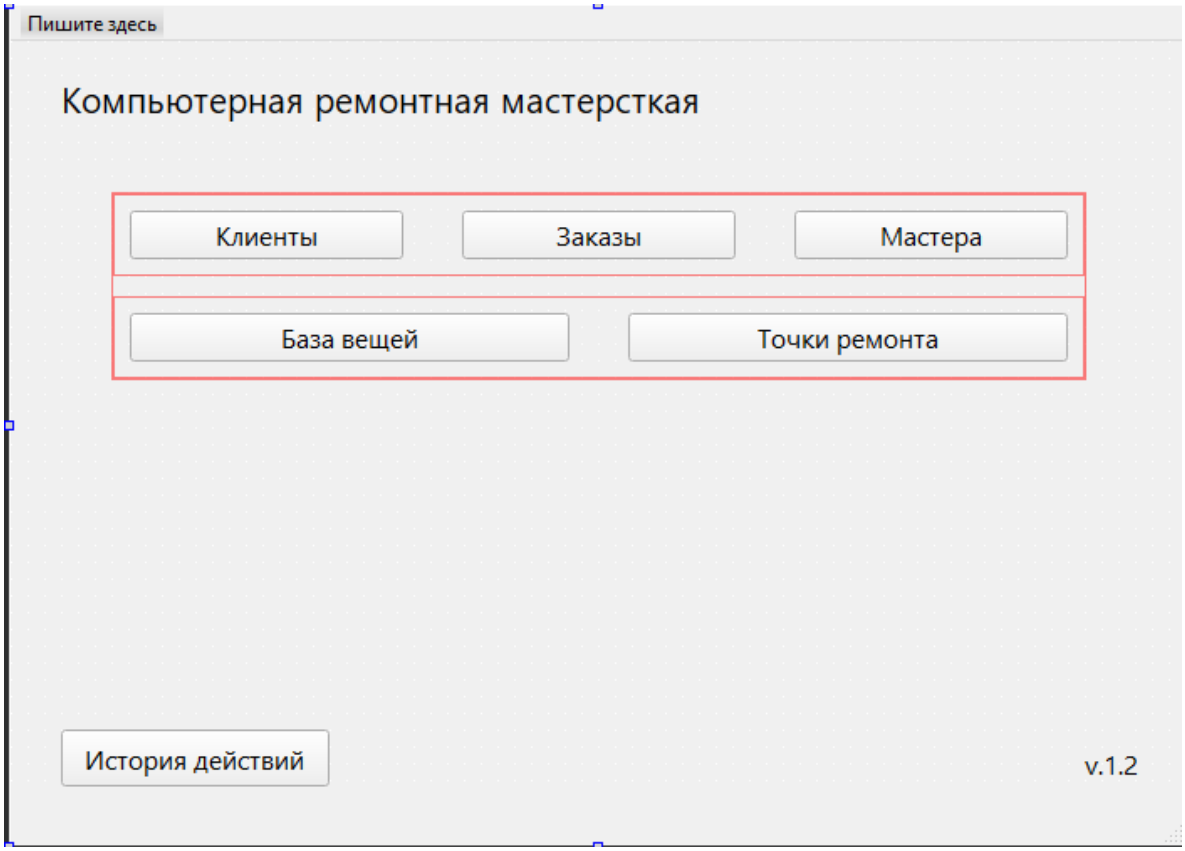


Рисунок 7 – Форма MainWindow

Clientsmenu – Меню, предназначенное для взаимодействия с сущностью клиент, в нашем случае добавление клиентов в систему. Форма состоит из:

- MainTitle – Qlabel – Поле с заголовком окна
- AddClient – QpushButton – Кнопка добавления клиента
- ClientTable – QtableView – Таблица с клиентами
- AdressLayout – QHBoxLayout – Компоновка поля ввода адреса по горизонтали
    - AdressEdit – QlineEdit – Поле ввода адреса
    - AdressLabel – Qlabel – Поле названия ввода
- FIOLayout – QHBoxLayout – Компоновка поля ввода ФИО по горизонтали
    - FIOEdit – QlineEdit – Поле ввода ФИО
    - FIOLabel – Qlabel – Поле названия ФИО
- PhoneLayout – QHBoxLayout – Компоновка поля ввода телефона по горизонтали
    - PhoneEdit – QlineEdit – Поле ввода телефона
    - PhoneLabel – Qlabel – Поле названия телефона

Форма представлена на рисунке 8.

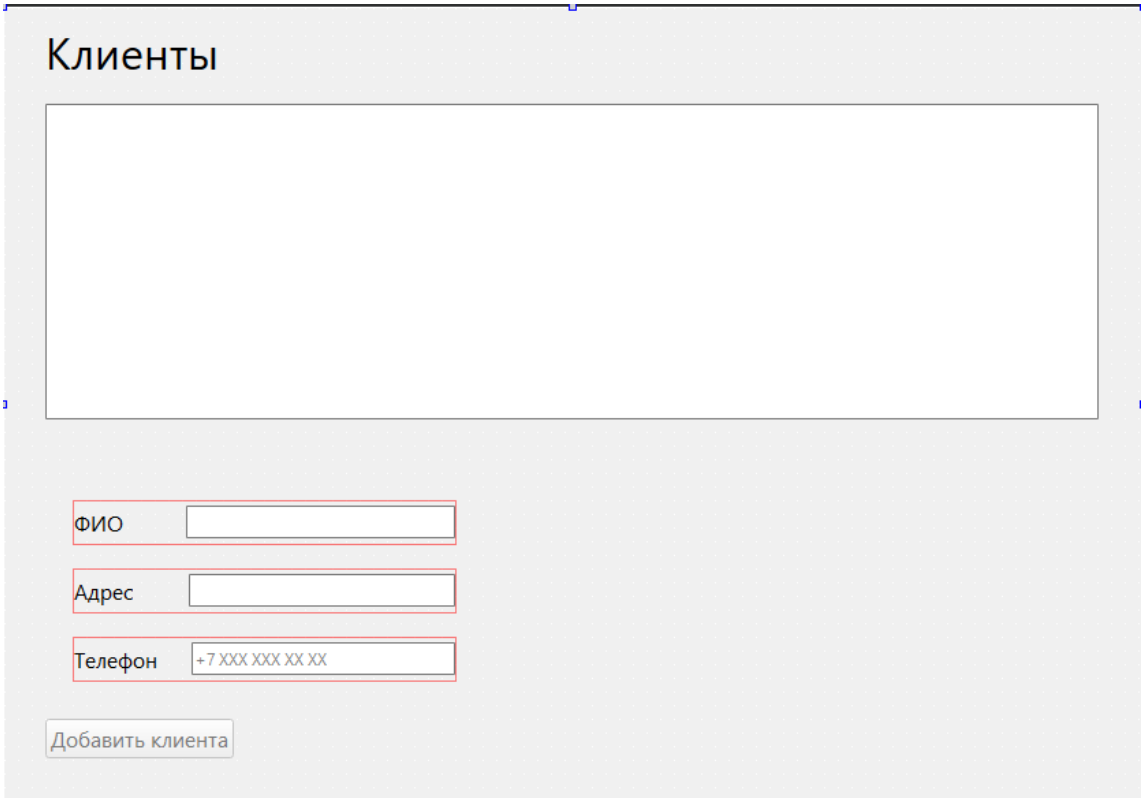


Рисунок 8 – Форма Clientsmenu

Mastersmenu – Меню, предназначенное для взаимодействия с сущностью мастер, а именно добавления мастеров и их увольнение. Форма состоит из:

- MainTitle – Qlabel – Поле с заголовком окна
- AddMaster – QpushButton – Кнопка добавления мастера
- RemoveMaster – QpushButton – Кнопка увольнения мастера
- MastersTable – QtableView – Таблица с мастерами
- PositionLayout – QHBoxLayout – Компоновка поля ввода должности по горизонтали
    - PositionEdit – QlineEdit – Поле выбора должности из списка
    - PositionLabel – Qlabel – Поле названия ввода
- FIOLayout – QHBoxLayout – Компоновка поля ввода ФИО по горизонтали
    - FIOEdit – QlineEdit – Поле ввода ФИО
    - FIOLabel – Qlabel – Поле названия ФИО
- RegLayout – QHBoxLayout – Компоновка поля выбора филиала по горизонтали
    - RegEdit – QlineEdit – Поле выбора филиала
    - RegLabel – Qlabel – Поле названия выбора

Форма представлена на рисунке 9.

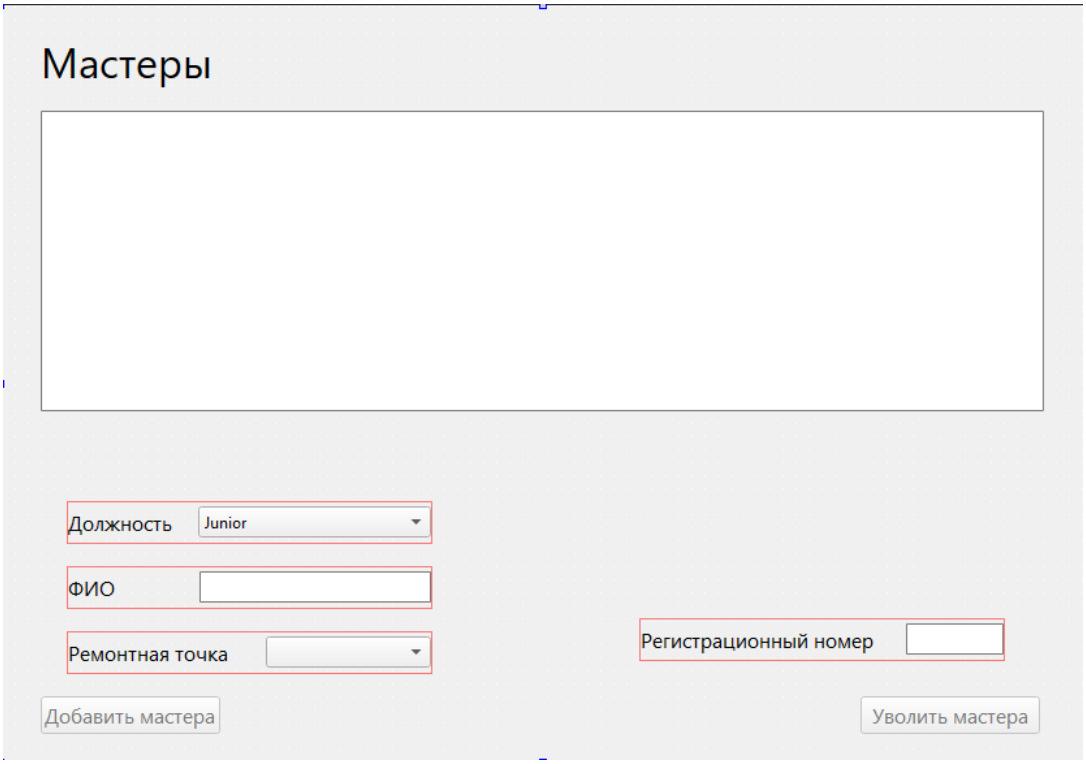


Рисунок 9 - Форма Mastersmenu

Ordersmenu – Меню, предназначенное для взаимодействия с сущностью заказ, в нашем случае добавление заказов, их выполнение и отмена. Форма состоит из:

- MainTitle – Qlabel – Поле с заголовком окна
- AddOrder – QpushButton – Кнопка добавления заказа
- RemoveOrder – QpushButton – Кнопка отмены заказа
- ExecuteOrder – QpushButton – Кнопка выполнения заказа
- MastersTable – QtableView – Таблица с заказами
- NumOfRowLayout– QHBoxLayout – Компоновка поля ввода номера строки по горизонтали
    - NumOfRowEdit – QlineEdit – Поле ввода
    - NumOfRowLabel – Qlabel – Поле названия
- RegNumClientLayout– QHBoxLayout – Компоновка поля ввода рег номера клиента по горизонтали
    - RegNumClientEdit – QlineEdit – Поле ввода
    - RegNumClientLabel – Qlabel – Поле названия
- RegNumMasterLayout– QHBoxLayout – Компоновка поля ввода рег номера мастера по горизонтали
    - RegNumMasterEdit – QlineEdit – Поле ввода
    - RegNumMasterLabel – Qlabel – Поле названия

- ThingLayout– QHBoxLayout – Компоновка поля ввода объекта ремонта по горизонтали
    - ThingEdit – QlineEdit – Поле ввода
    - ThingLabel – Qlabel – Поле названия

Форма представлена на рисунке 10.

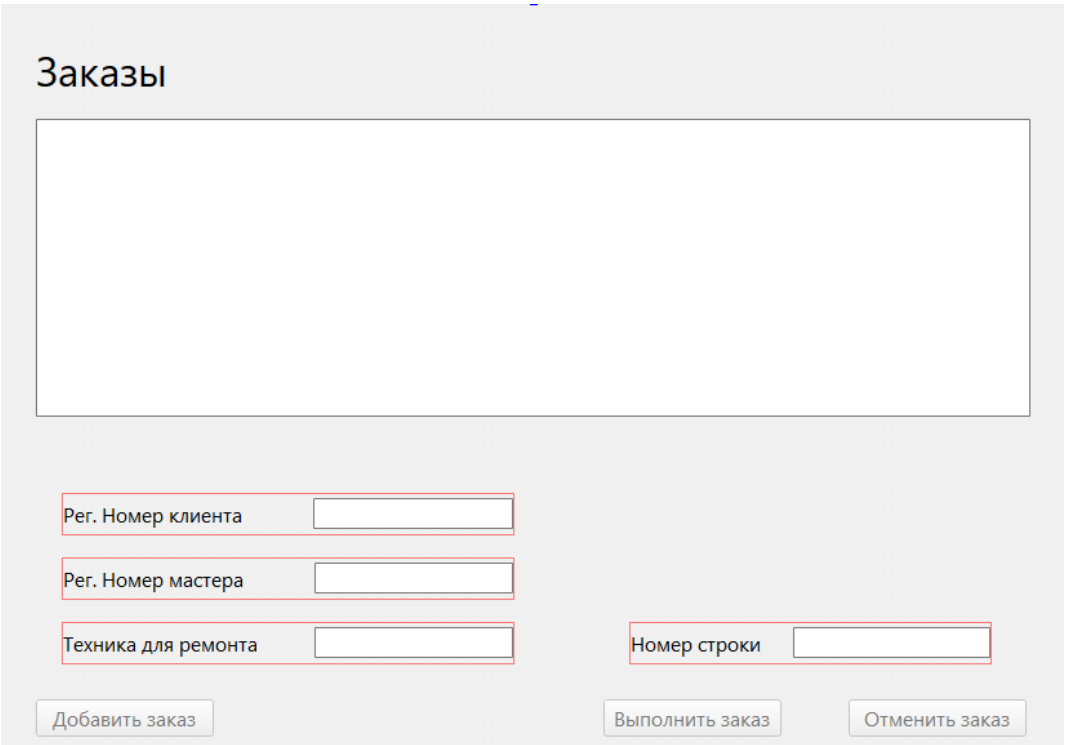


Рисунок 10 – Форма Ordersmenu

Placesmenu – Меню, предназначенное для взаимодействия с сущностью филиал, а именно его добавление и перевод в состояние активен/неактивен. Форма содержит:

- MainTitle – Qlabel – Поле с заголовком окна
- AddPlace – QpushButton – Кнопка добавления филиала
- EnablePlace – QpushButton – Кнопка возобновления работы
- StopPlace – QpushButton – Кнопка приостановки работы
- PlaceTable – QtableView – Таблица с филиалами
- AdressLayout – QHBoxLayout – Компоновка поля ввода адреса по горизонтали
    - AdressEdit – QlineEdit – Поле ввода
    - AdressLabel – Qlabel – Поле названия
- NameOfPlaceLayout – QHBoxLayout – Компоновка поля ввода названия по горизонтали
    - NameOfPlaceEdit – QlineEdit – Поле ввода
    - NameOfPlaceLabel – Qlabel – Поле названия

- PhoneNumLayout – QHBoxLayout – Компоновка поля ввода телефона по горизонтали
    - PhoneNumEdit – QlineEdit – Поле ввода
    - PhoneNumLabel – Qlabel – Поле названия
- NumOfRowLayout_1– QHBoxLayout – Компоновка поля номера строки по горизонтали
    - NumOfRowEdit_1– QlineEdit – Поле ввода
    - NumOfRowLabel_1– Qlabel – Поле названия

Форма представлена на рисунке 11.

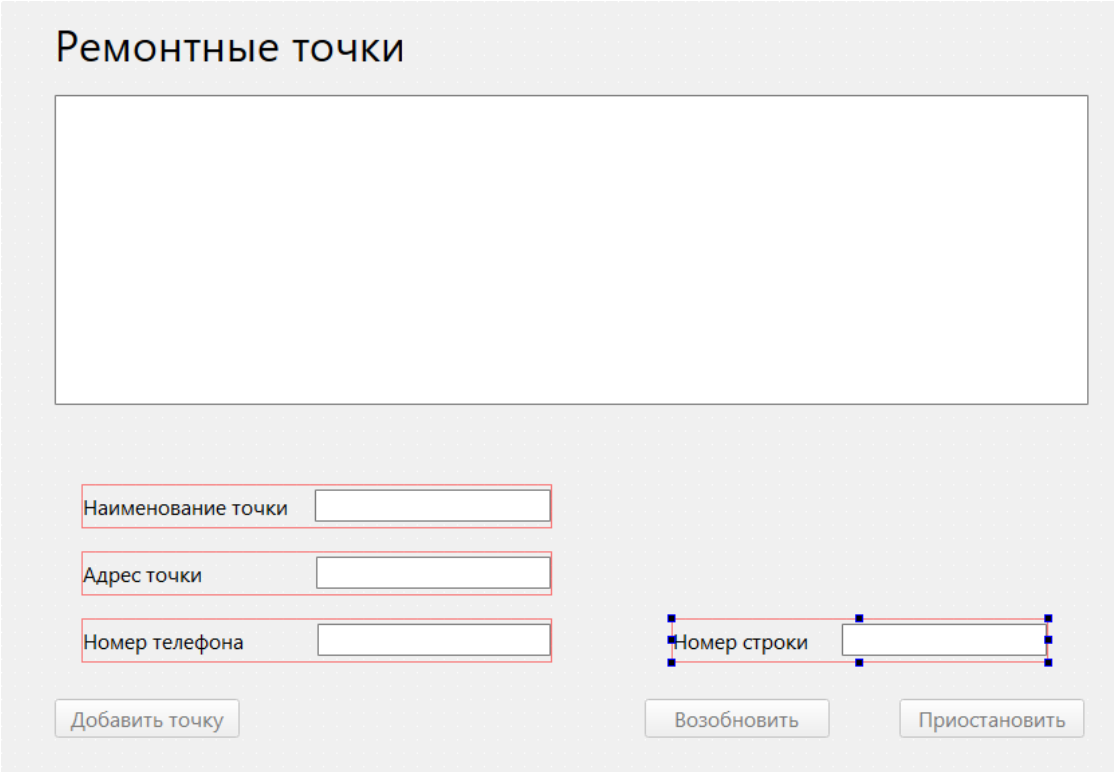


Рисунок 11 – Форма Placesmenu

Thingsmenu – Меню, предназначенное для взаимодействия с сущностью объект ременота, а именно добавление. Форма состоит из:

- MainTitle – Qlabel – Поле с заголовком окна
- AddThing – QpushButton – Кнопка добавления объекта
- ThingsTable – QtableView – Таблица с объектами
- PlaceLayout – QHBoxLayout – Компоновка поля выбора филиала по горизонтали
    - PlaceComboBox – QComboBox – Поле вывора
    - PlaceLabel – Qlabel – Поле названия

- ThingNameLayout – QHBoxLayout – Компоновка поля ввода названия объекта по горизонтали
    - ThingNameEdit – QlineEdit – Поле ввода
    - ThingNameLabel – Qlabel – Поле названия

Форма представлена на рисунке 12.

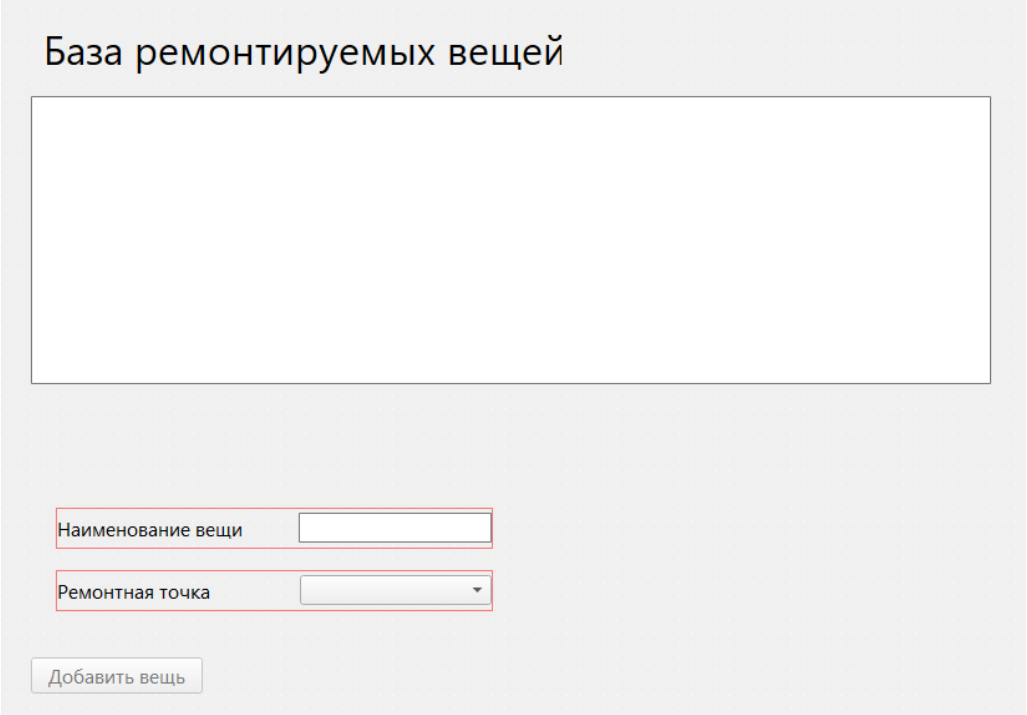


Рисунок 12 – Форма Thingsmenu

HistoryButton – Меню, предназначенное для просмотра истории действий. Форма состоит из:

- HistoryTable – QtableView – Таблица действий
- MainTable – Qlabel – Наименование окна

Форма представлена на рисунке 13.

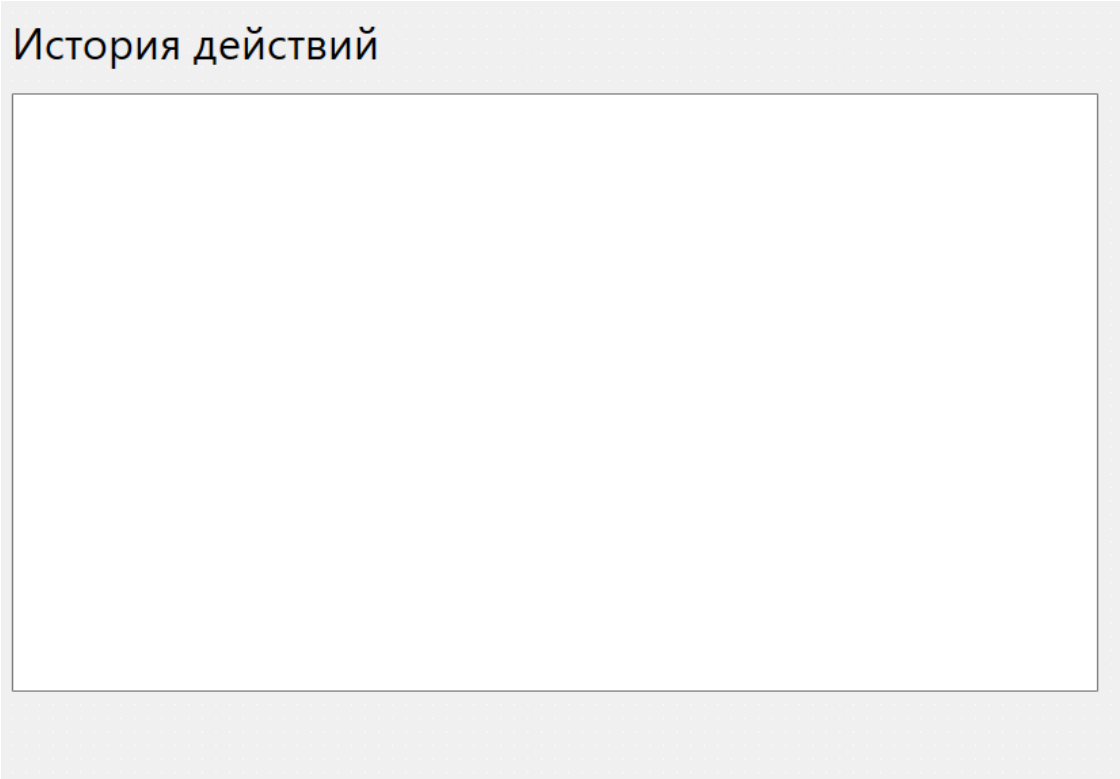


Рисунок 13 – Форма Historymenu

3.2 Реализация методов классов

Для каждой сущности в приложении существует соответствующий ей класс с полями и методами, определёнными на этапе 2.1 разработки классов сущностей. Далее будут описаны методы классов, каждой из сущностей:

- Клиент
  - Конструктор – Создаёт клиента на основе ФИО, адреса и номера телефона
  - Геттеры – возвращают соответствующие им поле класса
  - PhoneNumberToRegNumber – Перевод номера телефона в регистрационный номер по алгоритму: каждая цифра номера возводится в квадрат и складывается
- Мастер
  - Конструктор – Создаёт клиента на основе ФИО, должности и филиала
  - Геттеры – возвращают соответствующие им поле класса
  - SetOrder – Установка номера заказа, которым занят мастер. -1 для отсутствия заказа
  - SetStatus – Статус работы мастера: работает, уволен

    - GetOrderStatus – Возвращает номер заказа или “Нет” в случае его отсутствия
    - FIOnPosToRegCode – Перевод ФИО в регистрационный номер по алгоритму: Сложение всех букв ФИО путём перевода их в код юникода и добавление в начало: IN-, JN-, ML-, SN-, OT- в зависимости от должности: Intern, Junior, Middle, Senior, Other
- Заказ
    - Конструктор – Создаёт клиента на основе рег номеров клиента и мастера, а так же объекта ремонта
    - Геттеры – возвращают соответствующие им поле класса
    - GetStringStatusOfRepair – Преобразование статуса заказа в строку по алгоритму: InProcess-> В процессе, Cancelled-> Отменён ; Completed-> Выполнен
- Филиал
    - Конструктор – Создаёт клиента на основе имени филиала, адреса филиала, номера телефона филиала, статуса работы
    - Геттеры – возвращают соответствующие им поле класса
    - GetStringPlaceStatus - Преобразование статуса заказа в строку по алгоритму: InWork-> Работает, Cancelled-> Приостановлен
- Объект ремонта
    - Конструктор – Создаёт клиента на основе имени объекта и имени филиала
    - Геттеры – возвращают соответствующие им поле класса
    - GetThingStatusString - Преобразование статуса заказа в строку по алгоритму: InWork-> Работает, Cancelled-> Отсутствует
    - IsThingInOrder – Возвращает булево значение, участвует ли объект в заказе
    - SetOrder – Установка заказа, с которым связан объект

Далее для каждого класса интерфейса был разработан функционал, необходимый для выполнения требований, выявленным на этапе анализа предметной области. Далее описаны методы классов каждого из интерфейсов:

- Mainwindow
    - Конструктор – Инициализация всех окон в приложении, описанных ранее.
    - Геттеры – Геттеры соответствующих объектов класса интерфейса

- ClientsMenu
    - Конструктор – Установка базовых настроек таблицы клиентов и установка валидации ввода для полей
    - Деструктор – Очистка памяти от списка клиентов
    - SetModelHeader – Установить заголовок таблицы
    - AddNewClientToModel – Добавить клиента в таблицу
    - IsClientInList – Проверка наличия клиента в списке
    - RequestToAddClient - Запрос на добавление клиента в таблицу с предварительной проверкой конфликтов
- OrdersMenu
    - Конструктор – Установка базовых настроек таблицы заказов и установка валидации ввода для полей
    - Деструктор – Очистка памяти от списка заказов
    - GetSizeOfList – Получить размер списка заказов
    - SetModelHeader – Установить заголовок таблицы
    - ClearInputs – Очистить ввод
    - AddNewOrderToModel – Добавить заказ в таблицу
    - CancelOrder – Отменить заказ
    - GetOrderByNumber – Получить заказ по порядковому номеру
    - SetOrderAsCompleted – Установить заказ выполненым
    - InfoChanged - Управление кнопками в зависимости от введённых данных
    - RowNumberChanged - Управление кнопками в зависимости от введённых данных
- MastersMenu
    - Конструктор – Установка базовых настроек таблицы мастеров и установка валидации ввода для полей
    - Деструктор – Очистка памяти от списка мастеров
    - SetMasterBusyStatusByRegCode – Установить заказ мастеру
    - SetModelHeader – Установить заголовок таблицы
    - ClearInputs – Очистить ввод
    - AddNewMasterToModel – Добавить мастера в таблицу
    - UpdatePlaceList – Добавить филиал в список
    - FireMaster – Уволить мастера
    - IsMasterInList – Проверка на наличие мастера по рег коду

    - GetMasterByRegCode – Получить мастера по рег коду
    - RequestToAddMaster - Запрос на добавление мастера в таблицу с предварительной проверкой конфликтов
    - InfoChanged - Управление кнопками в зависимости от введённых данных
    - RowNumberChanged - Управление кнопками в зависимости от введённых данных
- ThingsMenu
    - Конструктор – Установка базовых настроек таблицы объектов
    - Деструктор – Очистка памяти от списка объектов
    - ClearInputs – Очистка полей ввода
    - SetOrderStatusByName – Установка статуса заказа по названию объекта
    - IsThingNameInList – Проверка на наличие объекта в списке
    - GetThingByName – Получить объект класса по названию
    - AddNewThingToModel – Добавить новый объект в таблицу
    - SetModelHeader – Установка заголовка таблицы
    - RequestToAddThing – Запрос на добавление объекта в таблицу с предварительной проверкой конфликтов
    - UpdatePlaceList – Добавить филиал в выпадающий список меню
    - InfoChanged – Управление кнопками в зависимости от введённых данных
- PlacesMenu
    - Конструктор – Установка базовых настроек таблицы клиентов и установка валидации ввода для полей
    - Деструктор – Очистка памяти от списка филиалов
    - ClearInputs – Очистка ввода
    - GetPlaceByNumber – Получить объект класса филиал по номеру в таблице
    - GetPlaceByName – Получить объект класса филиал по имени
    - SetPlaceStatusByName – Установка статуса филиала по его имени
    - SetModelHeader – Установить заголовок таблицы
    - AddNewPlaceToModel – Добавление филиала в таблицу
    - RequestToAddPlace – Запрос на добавление филиала в таблицу с предварительной проверкой конфликтов
    - IsPlaceNameInList – Проверка на наличие филиала по имени

    - IsPlaceAdressInList – Проверка на наличие филиала по адресу
    - EnablePlace – Изменить статус филиала на Работает
    - StopPlace – Изменить статус филиала на Приостановлен
    - InfoChanged - Управление кнопками в зависимости от введённых данных
    - RowNumberChanged - Управление кнопками в зависимости от введённых данных
- HistoryMenu
    - Конструктор – Установка базовых настроек таблицы действий
    - NewAction - Добавление нового действия в таблицу действий

Для реализации паттерна фасад был создан класс OrderManipulate, к которому обращаются другие классы при действиях с многозначной зависимостью. Методы класса описаны далее:

- SetRootWidget – Установка виджета, который имеет доступ ко всем окнам
- SendErrorMessage – Отправить диалоговое окно пользователю при ошибке
- AddOrder – Добавить заказ с проверкой всех зависимостей
- ExecuteOrder – Выполнить заказ с проверкой всех зависимостей
- RemoveOrder – Отменить заказ с проверкой всех зависимостей
- RemoveMaster – Уволить мастера с проверкой всех зависимостей
- NewAction – Функция добавления действия в таблицу истории действий

3.3 Разработка тестового приложения

На основе функционала вышеописанных методов разрабатывается приложение с функционалом, выявленным на этапе анализа предметной области. Основной задачей на данном этапе является связка всех окон приложения в единую структуру и добавление функциональности объектам в интерфейсе с помощью методов, определённых ранее.

Основной связью в приложении является паттерн Наблюдатель, структура которого описана в разработке классов. Паттерн реализован с помощью стандартной функции фреймворка connect, которая соединяет функции и сигналы между собой. Далее описаны сигналы каждого из интерфейсных окон:

- MainWindow
    - Сигналы на открытие меню при нажатии соответствующей кнопки в меню.
    - Сигналы на действия: добавить, выполнить, отменить заказ, уволить мастера, для перенаправления работы в класс OrderManipulate.

    - Сигналы при добавлении новых филиалов для обновления списков в меню мастеров и объектов.
- ClientsMenu
    - Сигнал при добавлении клиента
    - Сигналы при изменении полей ввода для включения/выключения кнопок
- OrdersMenu
    - Сигналы при изменении полей ввода для включения/выключения кнопок
    - Сигнал на добавление заказа
    - Сигнал на выполнение заказа
    - Сигнал на отмены заказа
- MastersMenu
    - Сигналы при изменении полей ввода для включения/выключения кнопок
    - Сигнал на добавление мастера
    - Сигнал на увольнение мастера
- ThingsMenu
    - Сигнал при изменении полей ввода для включения/выключения кнопок
    - Сигнал на добавление объекта
- PlacesMenu
    - Сигналы при изменении полей ввода для включения/выключения кнопок
    - Сигнал на добавление филиала
    - Сигнал на изменения статуса филиала на Работает
    - Сигнал на изменение статуса филиала на Приостановлен
- HistoryMenu
    - Сигналы не предусмотрены

С помощью сигналов и разработанных методов классов в итоге было разработано приложение, исполняющее все заявленные функциональные требования.

4. Тестирование

Для оценки работоспособности приложения проведём тестирования всех функциональных требований. Тестирование будет проведено в каждом из существующих окон, где присутствует функционал. Далее представлено тестирование приложения:

- MainWindow
    - Открытие окон через основное меню представлено на рисунке 14.

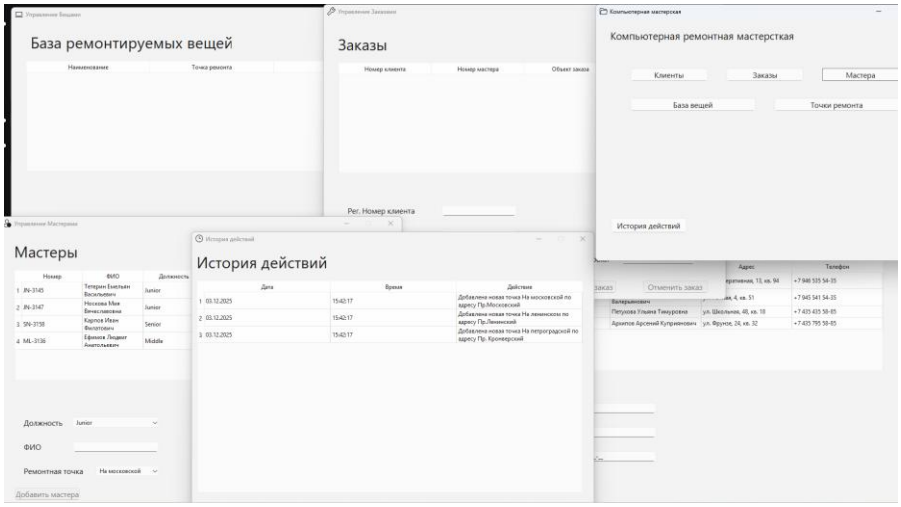


Рисунок 14 – Открытие окон

- HistoryMenu
    - Добавление действий в таблицу представлено на рисунке 15.

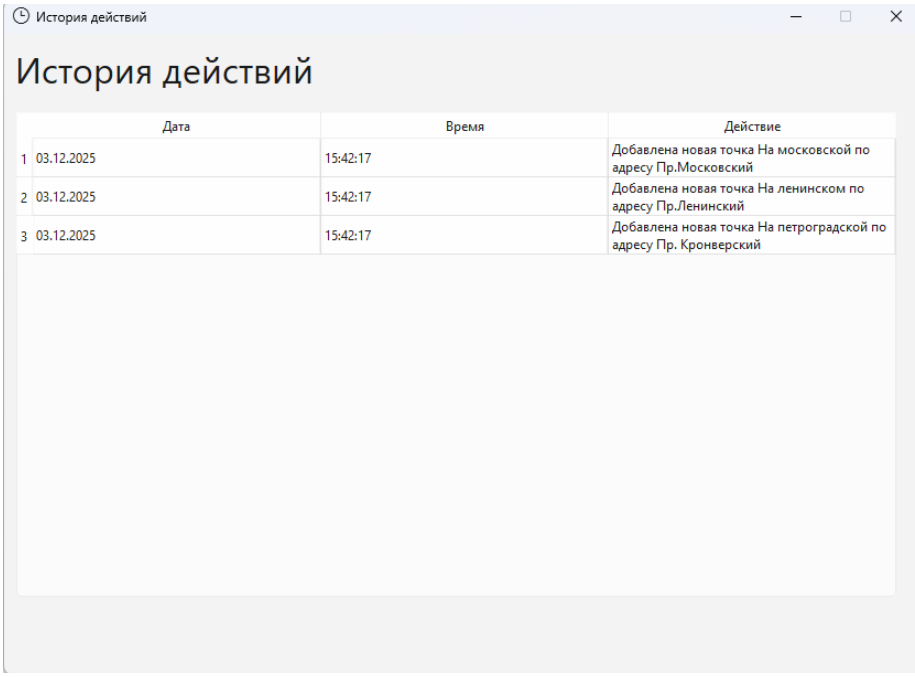


Рисунок 15 – Меню истории действий

- ClientsMenu

- Добавление клиента с уже существующим номером телефона на рисункуе 16.

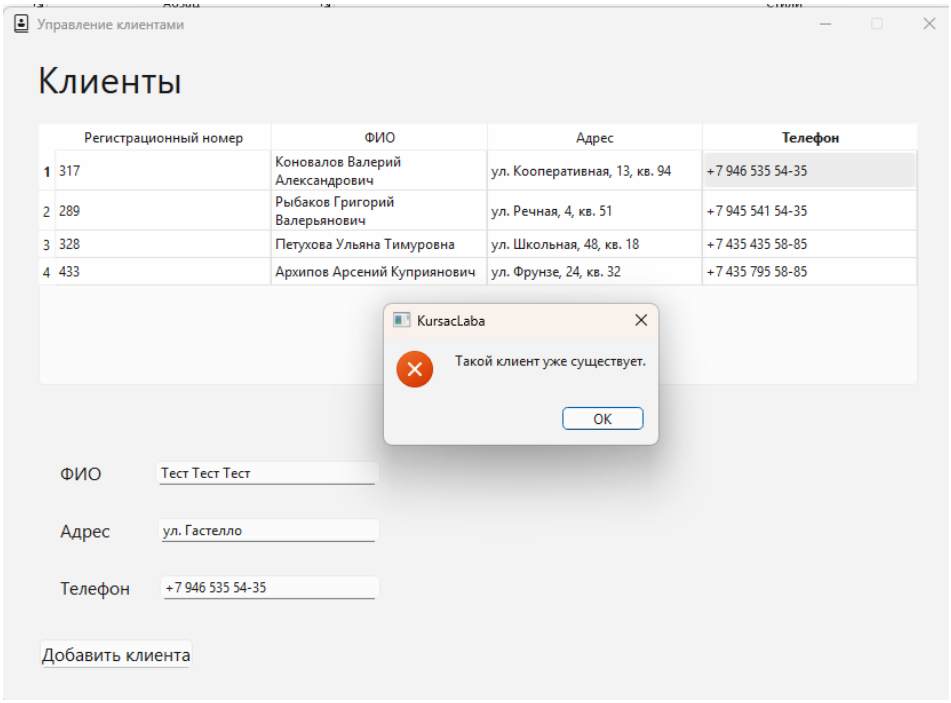


Рисунок 16 – Добавление клиента по существующему номеру

- Успешное добавление клиента на рисунке 17.

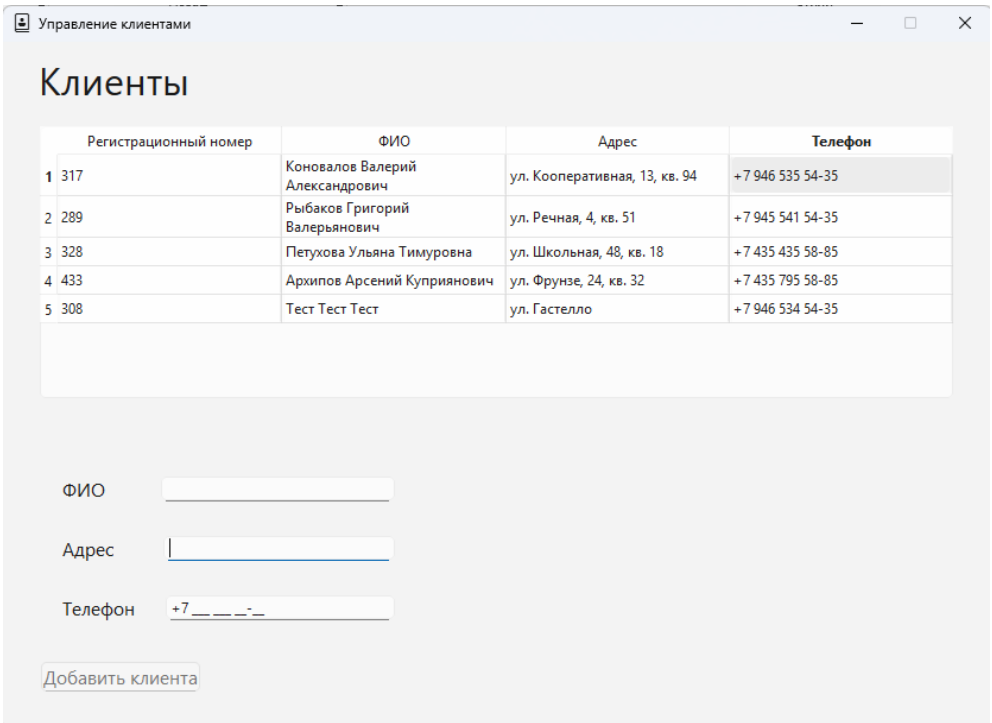


Рисунок 17 - Добавление клиента

- MastersMenu

- Добавление мастера с существующим ФИО и должностью на рисунке 18.

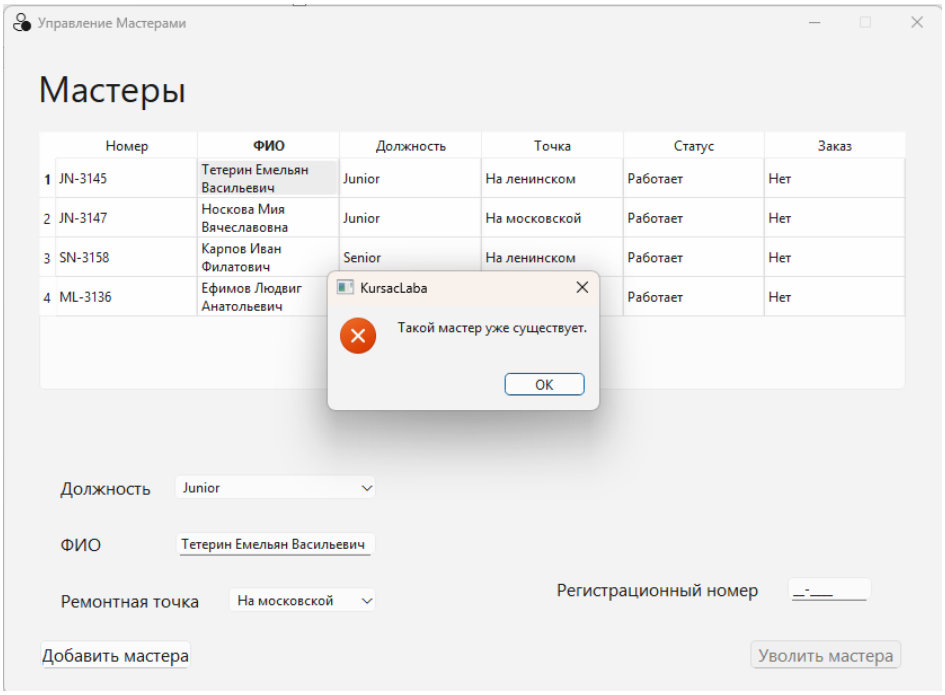


Рисунок 18 – Добавление мастера с существующей должностью и ФИО

- Добавление нового мастера на рисунке 19.

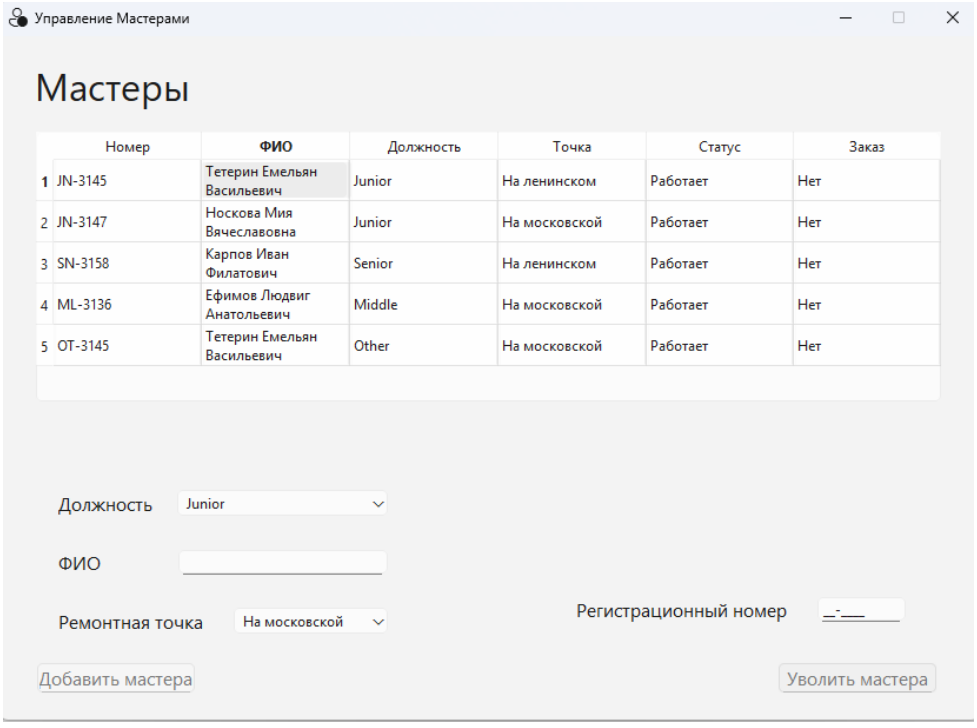


Рисунок 19 – Добавление мастера

- Увольнение мастера с невыполненным заказом на рисунке 20.

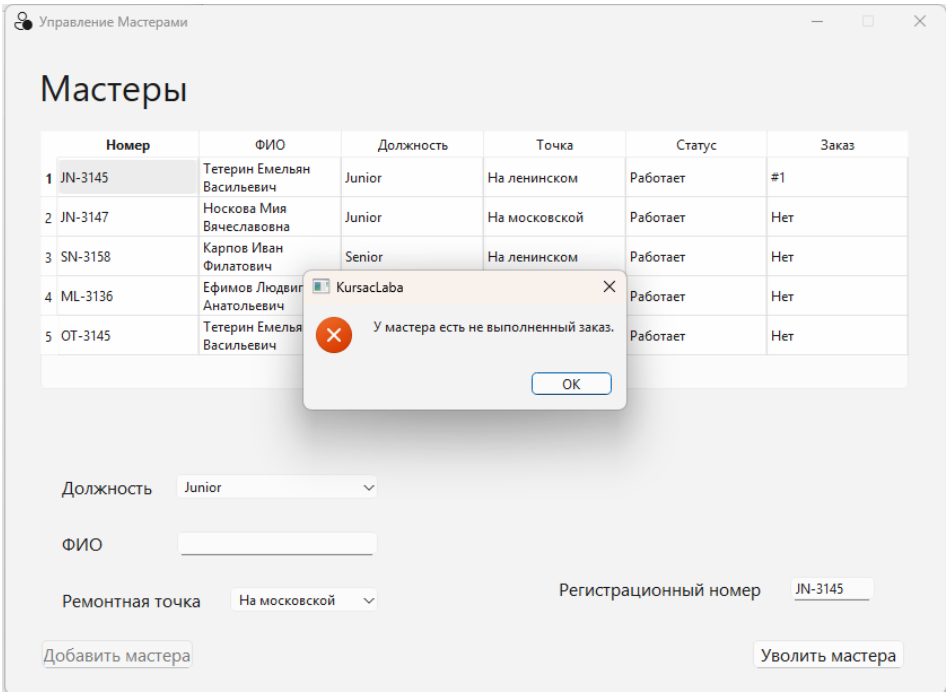


Рисунок 20 - Увольнение мастера с невыполненным заказом

- Увольнение мастера на рисунке 20

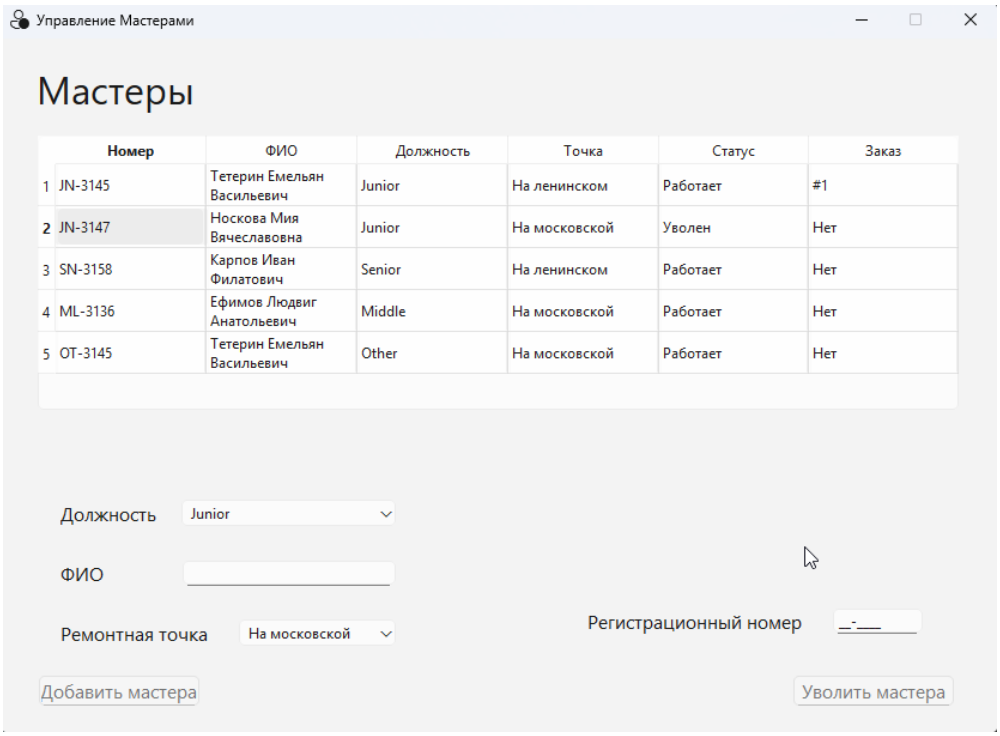


Рисунок 21 – Увольнение мастера

- ThingsMenu
  - Добавление объекта ремонта на рисунке 22

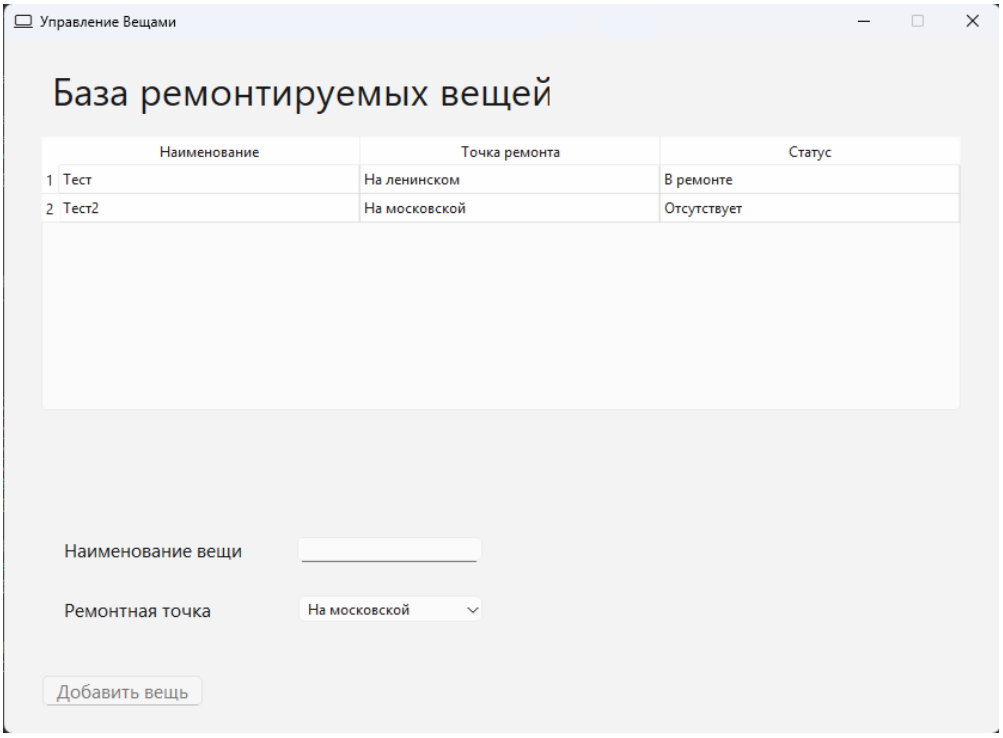


Рисунок 22 – Добавление объекта ремонта

- PlacesMenu
  - Добавление точки с существующим названием на рисунке 23

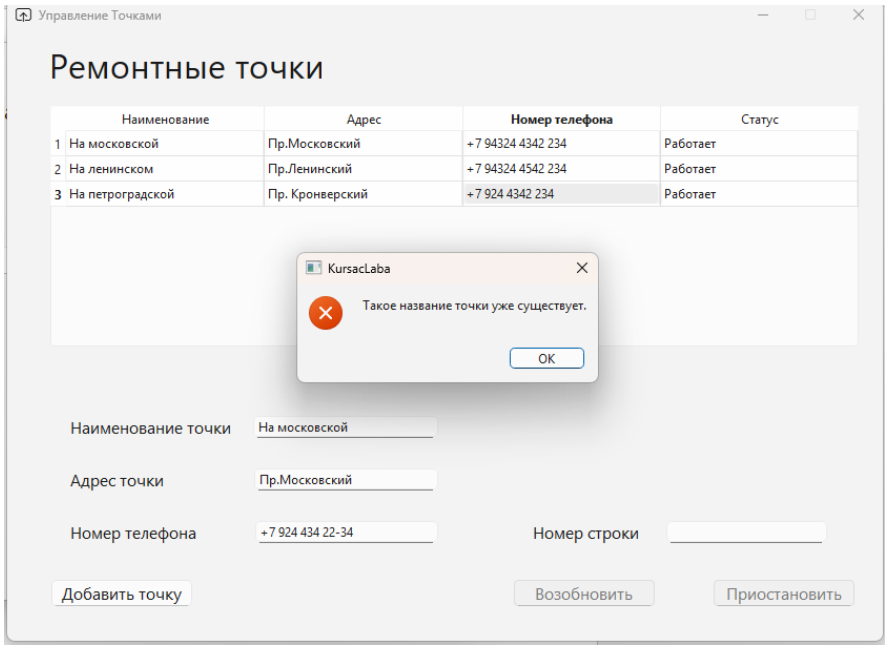


Рисунок 23 - Добавление филиала с существующим названием

  - Добавление филиала с существующим адресом на рисунке 24

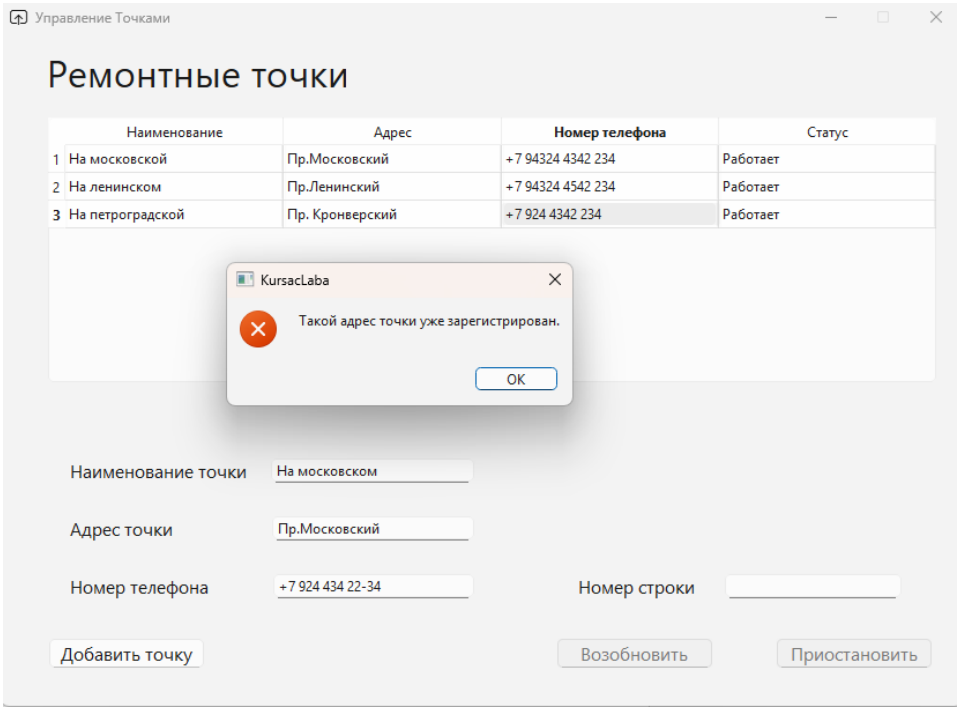


Рисунок 24 – Добавление филиала с существующим адресом

- Добавление филиала на рисунке 25

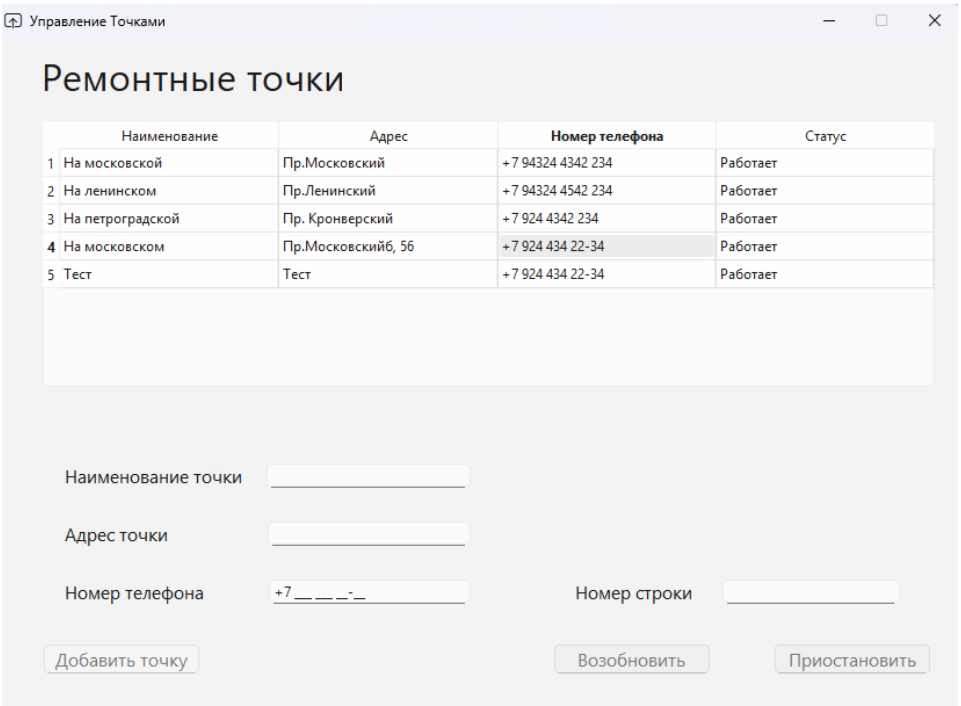


Рисунок 25 – Добавление филиала

- Приостановка деятельности филиала на рисунке 26

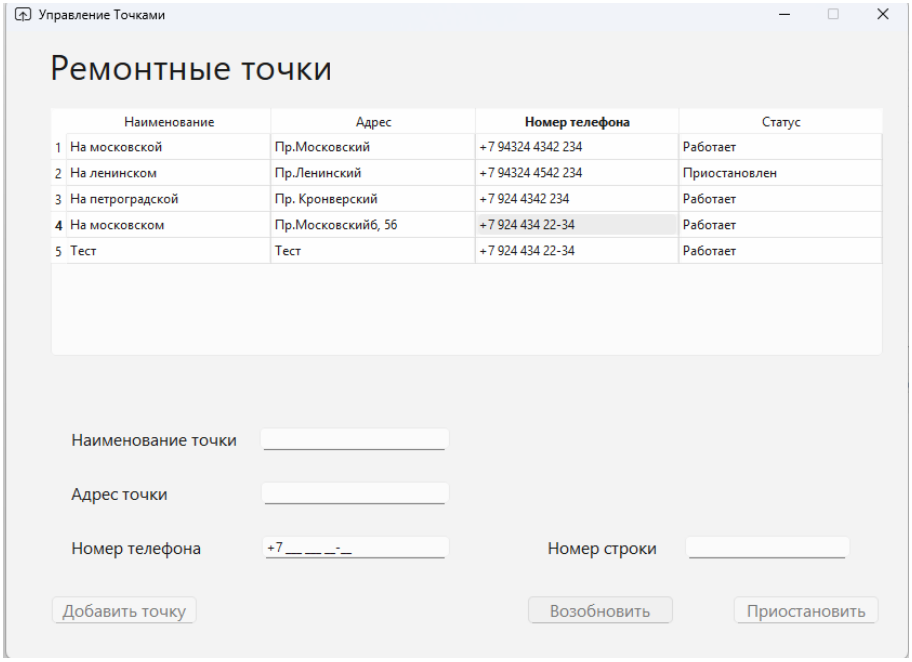


Рисунок 26 – Приостановка деятельности филиала

- Возобновление деятельности филиала

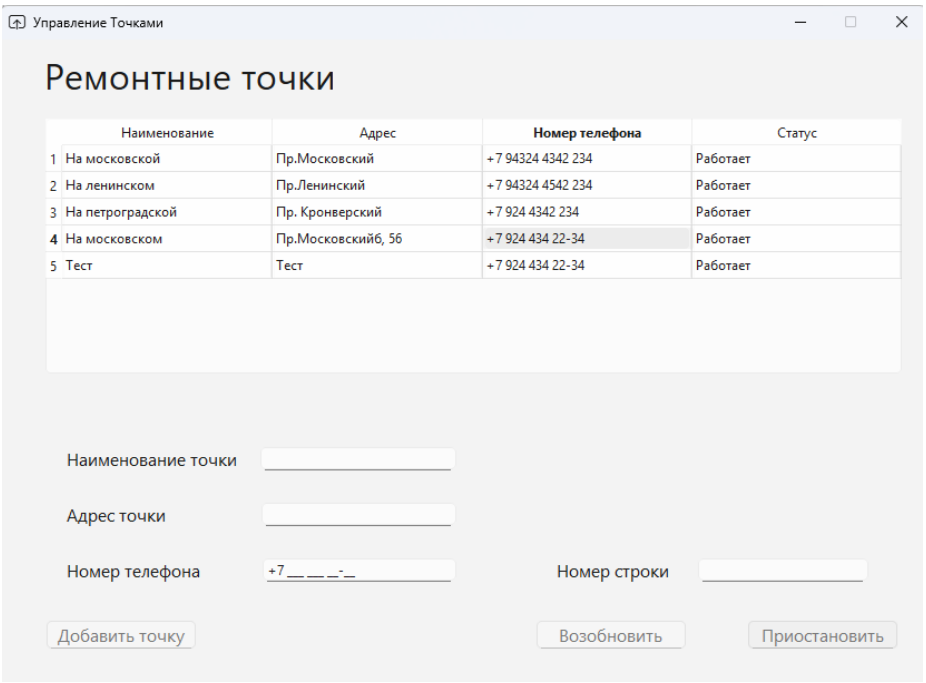


Рисунок 27 – Возобновление деятельности филиала

- OrdersMenu
  - Добавление заказа с несуществующим клиентом на рисунке 28

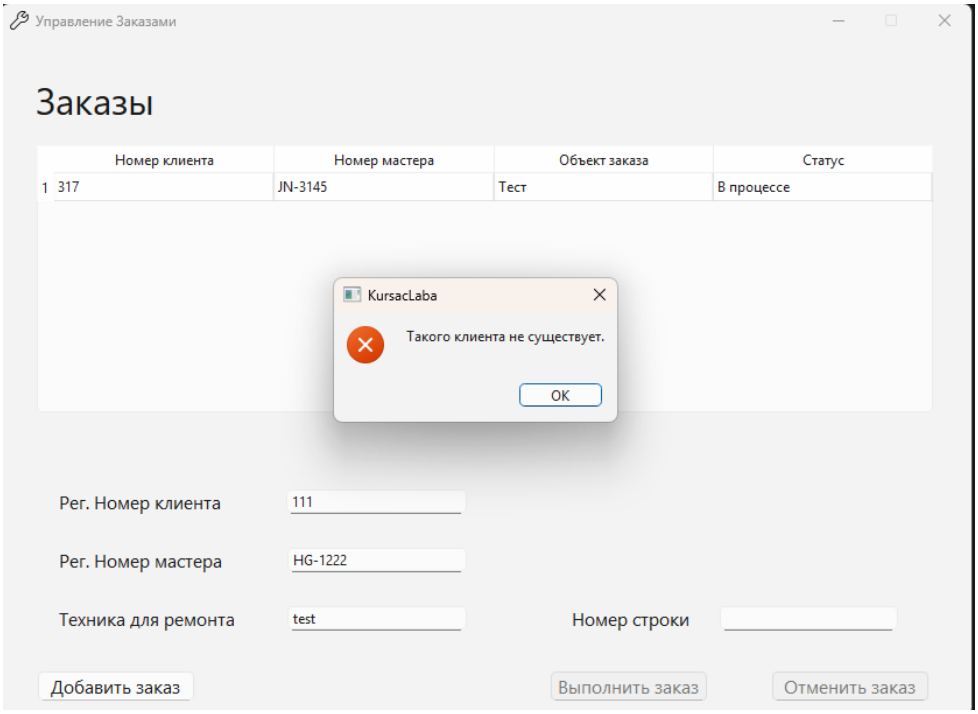


Рисунок 28 – Добавление заказа с несуществующим клиентом

- Добавление заказа с несуществующим мастером на рисунке 29

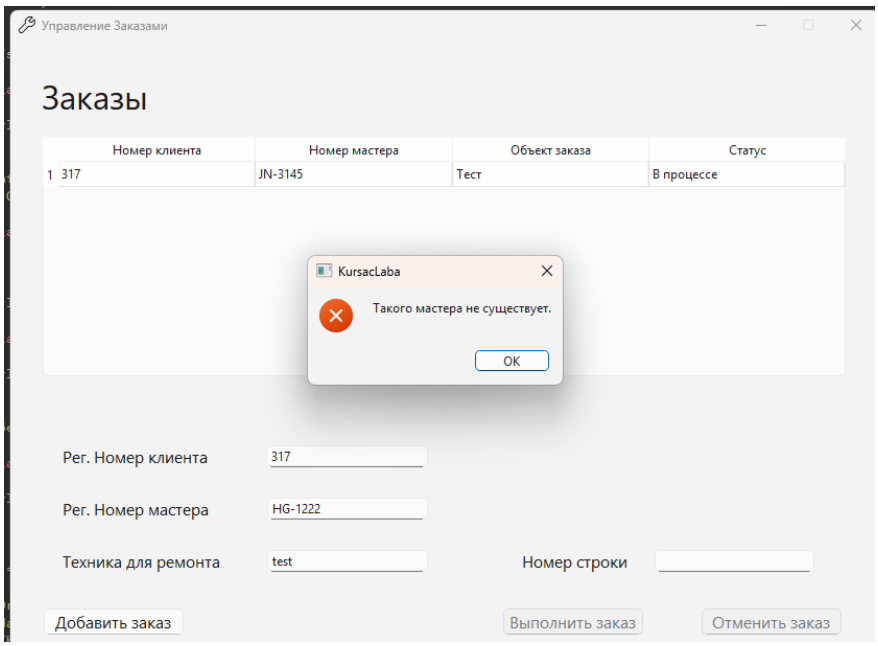


Рисунок 29 – Добавление заказа с несуществующим мастером

- Добавление заказа с не зарегистрированным объектом на рисунке 30

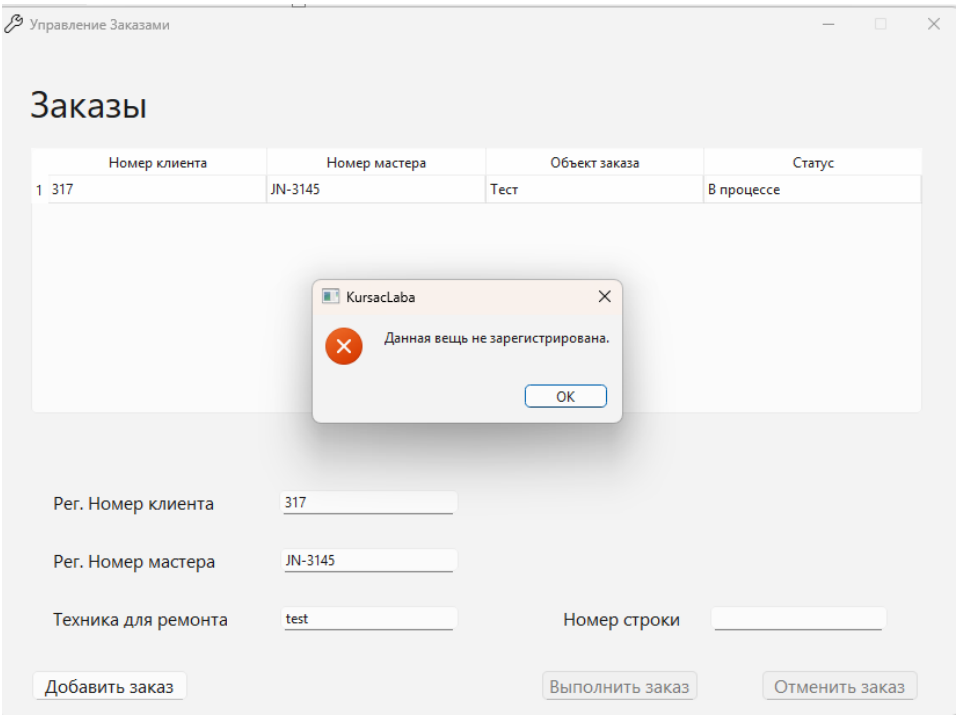


Рисунок 30 – Добавление заказа с не зарегистрированным объектом

- Добавление заказа на филиал, не принимающий заказы на рисунке 31

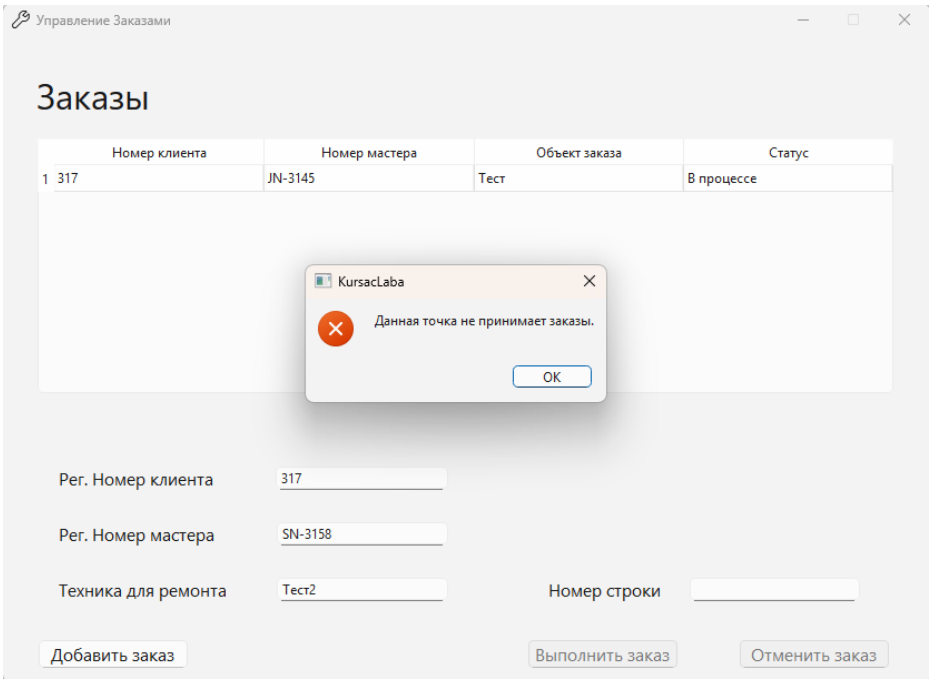


Рисунок 31 – Добавление заказа на филиал, не принимающий заказы

- Добавление заказа с объектом, который уже в заказе на рисунке 32

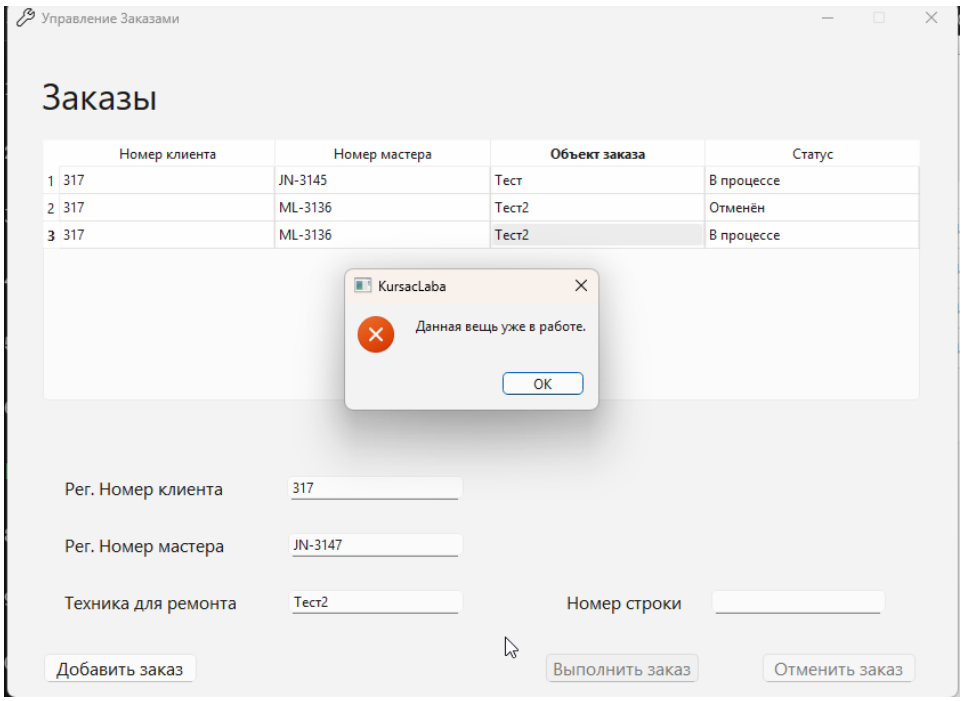


Рисунок 32 – Добавление заказа с объектом, который уже в заказе

- Добавление заказа с уволенным мастером на рисунке 33

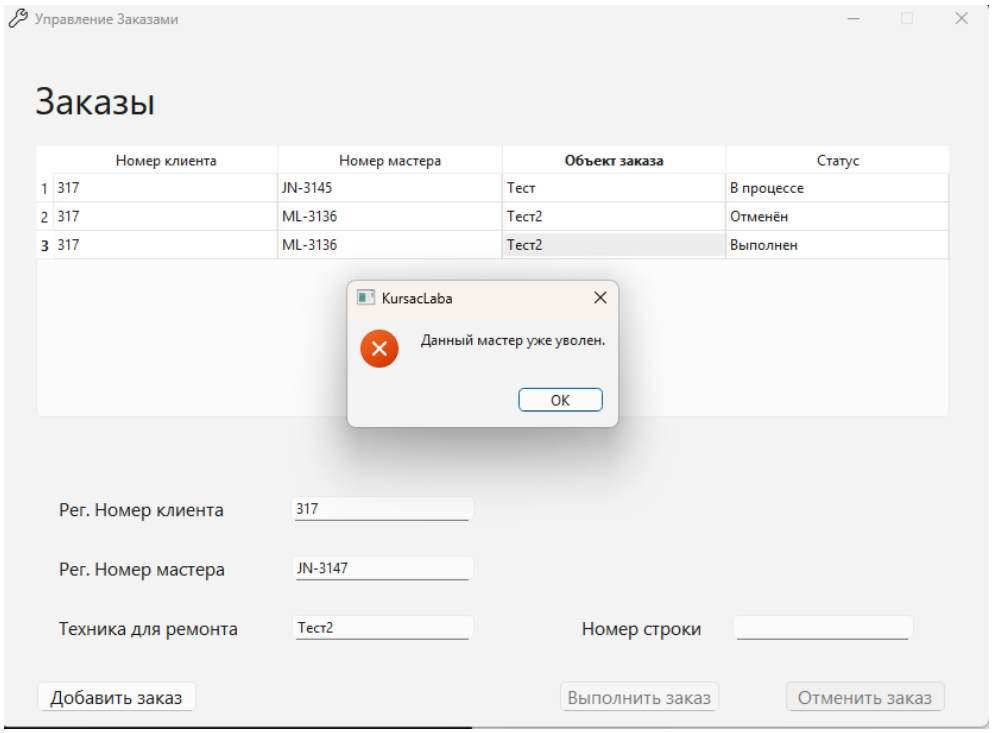


Рисунок 33 - Добавление заказа с уволенным мастером

- Добавление заказа с занятым мастером на рисунке 34

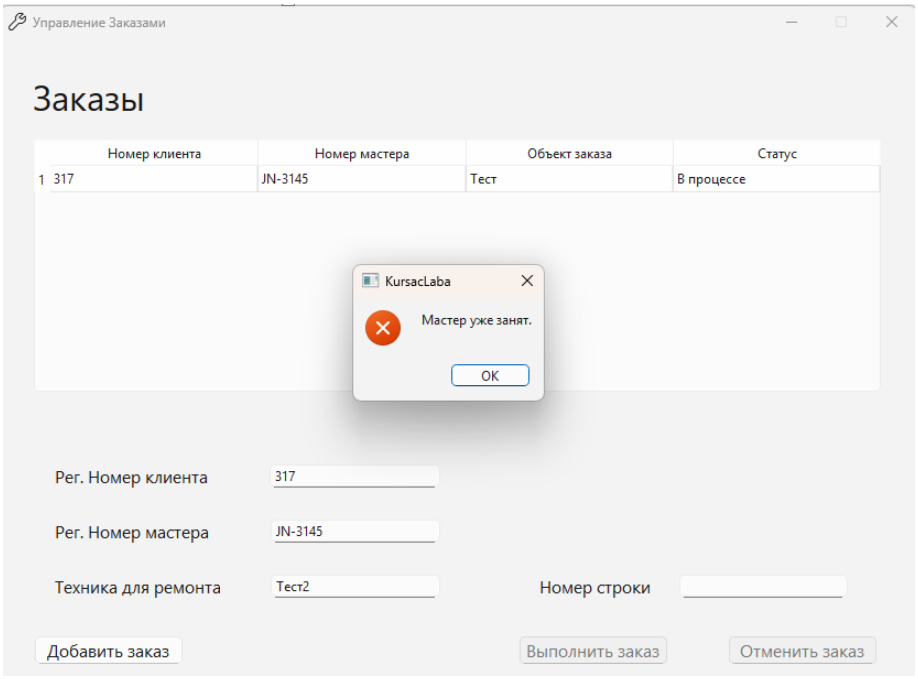


Рисунок 34 - Добавление заказа с занятым мастером

- Добавление заказа с разными филиалами мастера и объекта на рисунке 35

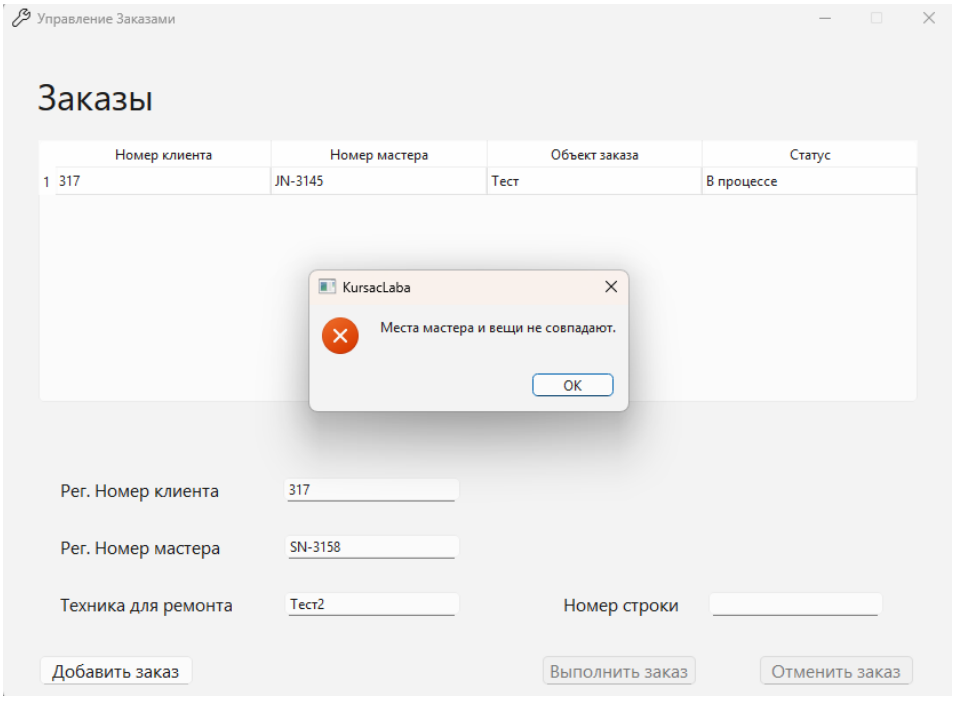


Рисунок 35 - Добавление заказа с разными филиалами мастера и объекта

Проведённое тестирование показывает, что приложение полностью соответствует поставленным функциональным требованиям.

ЗАКЛЮЧЕНИЕ

В процессе курсового проектирования было разработано приложение на C++ с использованием фреймворка QT. Данное приложение автоматизирует работу компьютерной мастерской и обеспечивает взаимодействие сущностей: клиент, мастер, заказ, филиал, объект ремонта. В процессе разработки были применены паттерны проектирования Фасад и Наблюдатель для построения структуры приложения. Исходный код представлен в приложении 1.

К преимуществам системы относится тот факт, что работа мастерской с помощью этой системы делает процесс более прозрачным и он подвержен меньшему количеству организационных ошибок. Тем не менее, приложение ограничено базовыми функциями организации заказов, в качестве рекомендаций к усовершенствованию системы можно выделить для добавления:

- Система уведомлений для клиентов/мастеров при изменении статуса заказа
- Система статистического анализа
- Возможность интеграции с другими сервисами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- Е.О. Шумова ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ. Методические указания к выполнению курсового проекта. - СПб: ГУАП, 2025
- C++ reference // C++ reference URL: https://en.cppreference.com/w/ (дата обращения: 9.10.2025).
- Qt Documentation // Qt Documentation URL: https://doc.qt.io/ (дата обращения: 9.10.2025).

ПРИЛОЖЕНИЕ 1. Исходный код

Client.h

```
#ifndef CLIENT_H
#define CLIENT_H

#include <QObject>

class Client : public QObject
{
   Q_OBJECT
public:
   explicit Client(QObject *parent = nullptr, QString FIO = "", QString Adress="", QString
PhoneNumber="");
   static int PhoneNumberToRegNumber(QString);

   int GetRegNumber();
   QString GetFIO();
   QString GetAdress();
   QString GetPhoneNumber();

private:
   int RegNumber;
   QString FIO;
   QString Adress;
   QString PhoneNumber;
};

#endif // CLIENT_H
```

Clientsmenu.h

```
#ifndef CLIENTSMENU_H
#define CLIENTSMENU_H

#include "client.h"
#include "qstandarditemmodel.h"
#include <QWidget>

namespace Ui {
class ClientsMenu;
}

class ClientsMenu : public QWidget
{
   Q_OBJECT

public:
   explicit ClientsMenu(QWidget *parent = nullptr);
   ~ClientsMenu();
   bool IsClientInList(int);
   void ClearInputs();
   void RemoveClientFromModel(int);
```

```
private:
    Ui::ClientsMenu* ui;
    int NumOfClients;
    QList<Client*> ClientsList;
    QStandardItemModel* model;

    void SetModelHeader();
    void AddNewClientToModel(Client*);


private slots:
    void RequestToAddClient();
    void InfoChanged();
};

#endif // CLIENTSMENU_H
```

Historymenu.h

```
#ifndef HISTORYMENU_H
#define HISTORYMENU_H

#include "qstandarditemmodel.h"
#include <QWidget>

namespace Ui {
class HistoryMenu;
}

class HistoryMenu : public QWidget
{
    Q_OBJECT

public:
    explicit HistoryMenu(QWidget *parent = nullptr);
    ~HistoryMenu();

    QStandardItemModel* model;

    void NewAction(QString);

private:
    Ui::HistoryMenu *ui;
};

#endif // HISTORYMENU_H
```

Mainwindow.h

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
```

```
#include "clientsmenu.h"
#include "mastersmenu.h"
#include "ordersmenu.h"
#include "placesmenu.h"
#include "thingsmenu.h"
#include "historymenu.h"

QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;
}
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

    ClientsMenu* GetClientsMenu();
    OrdersMenu* GetOrdersMenu();
    MastersMenu* GetMastersMenu();
    ThingsMenu* GetThingsMenu();
    PlacesMenu* GetPlacesMenu();

private:
    Ui::MainWindow* ui;
    ClientsMenu* CM;
    OrdersMenu* OM;
    MastersMenu* MM;
    ThingsMenu* TM;
    PlacesMenu* PM;

    HistoryMenu* HM;

signals:
    void NewAction(QString);
};
#endif // MAINWINDOW_H

Master.h
#ifndef MASTER_H
#define MASTER_H

#include <QObject>

class Master : public QObject
{
    Q_OBJECT
public:
```

```
    enum class MasterStatus
    {
        InWork,
        Fired
    };

    explicit Master(QObject *parent = nullptr, QString FIO="", QString Position="", QString
Place="");
    static QString FIOnPosToRegCode(QString, QString);

    void SetOrder(int);
    void SetStatus(bool);

    QString GetRegCode();
    QString GetFIO();
    QString GetPosition();
    QString GetPlaceName();
    MasterStatus GetStatus();
    QString GetStatusString();
    bool IsBusy();
    QString GetOrderStatus();

private:
    QString RegCode;
    QString FIO;
    QString Position;
    QString Place;
    MasterStatus Status;
    bool MasterBusy;
    int OrderNumber;
};

#endif // MASTER_H
```

Mastersmenu.h

```
#ifndef MASTERSMENU_H
#define MASTERSMENU_H

#include "qstandarditemmodel.h"
#include "master.h"
#include <QWidget>

namespace Ui {
class MastersMenu;
}

class MastersMenu : public QWidget
{
    Q_OBJECT

public:
    explicit MastersMenu(QWidget *parent = nullptr);
```

```
    ~MastersMenu();
    bool IsMasterInList(QString);
    void SetMasterBusyStatusByRegCode(QString, int);
    Master* GetMasterByRegCode(QString);
    void ClearInputs();
    void FireMaster(QString);


signals:
    void RequestToRemoveMaster(QString);

private:
    Ui::MastersMenu* ui;
    int NumOfMasters;
    QList<Master*> MasterList;
    QStandardItemModel* model;

    void SetModelHeader();
    void AddNewMasterToModel(Master*);

public slots:
    void UpdatePlaceList(QString);

private slots:
    void InfoChanged();
    void RegNumberChanged();
    void RequestToAddMaster();
};

#endif // MASTERSMENU_H

Order.h
#ifndef ORDER_H
#define ORDER_H

#include <QObject>

class Order : public QObject
{
    Q_OBJECT
public:
    enum class RepairStatus
    {
        InProcess,
        Cancelled,
        Completed
    };

    explicit Order(QObject *parent = nullptr, int ClientRegNumber = 0,
                   QString MasterRegCode = "", QString ThingToRepair = "");
    void SetOrderNumber(int);
    void SetRepairStatus(RepairStatus);
```

```
    int GetClientRegNumber();
    QString GetMasterRegCode();
    QString GetThingToRepair();
    RepairStatus GetStatusOfRepair();
    QString GetStringStatusOfRepair();

private:
    int ClientRegNumber;
    QString MasterRegCode;
    QString ThingToRepair;
    RepairStatus StatusOfRepair;
};

#endif // ORDER_H
```

OrderManipulate.h

```
#ifndef ORDERMANIPULATE_H
#define ORDERMANIPULATE_H

#include "mainwindow.h"
#include "order.h"
#include <QWidget>

class OrderManipulate : public QWidget
{
    Q_OBJECT
public:
    explicit OrderManipulate(QWidget *parent = nullptr);
    static void SetRootWidget(MainWindow*);
    static void SendErrorMessage(QString);
    static void AddOrder(Order*);
    static void ExecuteOrder(int);
    static void RemoveOrder(int);
    static void RemoveMaster(QString);
    static void NewAction(QString);

private:
    static MainWindow* RootWidget;
};

#endif // ORDERMANIPULATE_H
```

Ordersmenu.h

```
#ifndef ORDERSMENU_H
#define ORDERSMENU_H


#include "order.h"
#include "qstandarditemmodel.h"
#include <QWidget>
```

```
namespace Ui {
class OrdersMenu;
}

class OrdersMenu : public QWidget
{
   Q_OBJECT

public:
   explicit OrdersMenu(QWidget *parent = nullptr);
   ~OrdersMenu();

   void AddNewOrderToModel(Order*);
   void CancelOrder(int);
   Order* GetOrderByNumber(int);
   void SetOrderAsCompleted(int);
   int GetSizeOfList();
   void ClearInputs();

signals:
   void RequestToAddOrder(Order*);
   void RequestToExecuteOrder(int);
   void RequestToRemoveOrder(int);

private:
   Ui::OrdersMenu* ui;
   int NumOfOrders;
   QList<Order*> OrderList;
   QStandardItemModel* model;

   void SetModelHeader();

private slots:
   void InfoChanged();
   void RowNumberChanged();
};

#endif // ORDERSMENU_H
```

Place.h

```
#ifndef PLACE_H
#define PLACE_H

#include <QObject>

class Place : public QObject
{
   Q_OBJECT
public:
   enum class PlaceStatus
   {
```

```
        InWork,
        Suspended
    };

    explicit Place(QObject *parent = nullptr, QString PlaceName = "", QString PlaceAdress = "",
QString PhoneNumber = "", PlaceStatus Status = PlaceStatus::InWork);

    QString GetPlaceName();
    QString GetPlaceAdress();
    QString GetPhoneNumber();
    PlaceStatus GetPlaceStatus();
    QString GetStringPlaceStatus();

    void EnablePlace();
    void DisablePlace();

private:
    QString PlaceName;
    QString PlaceAdress;
    QString PhoneNumber;
    PlaceStatus Status;

};

#endif // PLACE_H
```

Placesmenu.h

```
#ifndef PLACESMENU_H
#define PLACESMENU_H

#include "place.h"
#include "qstandarditemmodel.h"
#include <QWidget>

namespace Ui {
class PlacesMenu;
}

class PlacesMenu : public QWidget
{
    Q_OBJECT

public:
    explicit PlacesMenu(QWidget *parent = nullptr);
    ~PlacesMenu();

    void ClearInputs();
    Place* GetPlaceByNumber(int);
    Place* GetPlaceByName(QString);
    void SetPlaceStatusByName(QString, bool);

private:
```

```
    Ui::PlacesMenu *ui;
    int NumOfPlaces;
    QList<Place*> PlacesList;
    QStandardItemModel* model;

    void SetModelHeader();
    void AddNewPlaceToModel(Place*);
    bool IsPlaceNameInList(QString);
    bool IsPlaceAdressInList(QString);

    void EnablePlace();
    void StopPlace();

private slots:
    void RequestToAddPlace();
    void InfoChanged();
    void RowNumberChanged();

signals:
    void PlaceAdded(QString);
};

#endif // PLACESMENU_H
```

Thing.h

```
#ifndef THING_H
#define THING_H

#include <QObject>

class Thing : public QObject
{
    Q_OBJECT
public:
    enum class ThingStatus
    {
        InWork,
        None
    };

    explicit Thing(QObject *parent = nullptr, QString ThingName="", QString PlaceName="");

    QString GetThingName();
    QString GetPlaceName();
    QString GetThingStatusString();
    bool IsThingInOrder();

    void SetOrder(int);
private:
    QString ThingName;
    QString PlaceName;
    ThingStatus Status;
```

```
    bool ThingInOrder;
    int OrderNumber;

};

#endif // THING_H
```

ThingsMenu.h

```
#ifndef THINGSMENU_H
#define THINGSMENU_H

#include "qstandarditemmodel.h"
#include "thing.h"
#include <QWidget>

namespace Ui {
class ThingsMenu;
}

class ThingsMenu : public QWidget
{
    Q_OBJECT

public:
    explicit ThingsMenu(QWidget *parent = nullptr);
    ~ThingsMenu();

    void ClearInputs();
    void SetOrderStatusByName(QString, int);
    bool IsThingNameInList(QString);
    Thing* GetThingByName(QString);

private:
    Ui::ThingsMenu *ui;
    int NumOfThings;
    QList<Thing*> ThingsList;
    QStandardItemModel* model;

    void AddNewThingToModel(Thing*);
    void SetModelHeader();

public slots:
    void RequestToAddThing();
    void UpdatePlaceList(QString);
    void InfoChanged();
};

#endif // THINGSMENU_H
```

Client.cpp

```
#include "client.h"
```

```
Client::Client(QObject *parent, QString FIO, QString Adress, QString PhoneNumber)
    : QObject{parent}
{
    this->FIO = FIO;
    this->Adress = Adress;
    this->PhoneNumber = PhoneNumber;

    this->RegNumber = Client::PhoneNumberToRegNumber(PhoneNumber);
}

QString Client::GetFIO()
{
    return this->FIO;
}

QString Client::GetAdress()
{
    return this->Adress;
}

QString Client::GetPhoneNumber()
{
    return this->PhoneNumber;
}

int Client::GetRegNumber()
{
    return this->RegNumber;
}

int Client::PhoneNumberToRegNumber(QString PhoneNumber)
{
    QString NumberToSum = PhoneNumber.replace( " ", "" ).remove('+');
    int OutSum = 0;
    for (auto number : std::as_const(NumberToSum) )
    {
        OutSum += pow(number.digitValue(), 2);
    }
    return OutSum;
}
```

Clientsmenu.cpp

```
#include "clientsmenu.h"
#include "client.h"
#include "ui_clientsmenu.h"
#include "ordermanipulate.h"
#include <QStandardItemModel>

ClientsMenu::ClientsMenu(QWidget *parent)
    : QWidget(parent)
    , ui(new Ui::ClientsMenu)
```

```
{
    ui->setupUi(this);
    this->setWindowFlag(Qt::Window);

    this->NumOfClients = 0;
    auto ClientTable = ui->ClientTable;
    ClientTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
    ClientTable->horizontalHeader()->setSectionResizeMode(QHeaderView::Stretch);

    model = new QStandardItemModel(0, 4, this);
    SetModelHeader();

    ClientTable->setModel(model);
    ClientTable->setWordWrap(true);
    ui->PhoneEdit->setInputMask("+7 999 999 99-99;_");

    AddNewClientToModel(new Client(nullptr, "Коновалов Валерий Александрович", "ул.
Кооперативная, 13, кв. 94", "+7 946 535 54-35"));
    AddNewClientToModel(new Client(nullptr, "Рыбаков Григорий Валерьянович", "ул. Речная,
4, кв. 51", "+7 945 541 54-35"));
    AddNewClientToModel(new Client(nullptr, "Петухова Ульяна Тимуровна", "ул. Школьная,
48, кв. 18", "+7 435 435 58-85"));
    AddNewClientToModel(new Client(nullptr, "Архипов Арсений Куприянович", "ул. Фрунзе,
24, кв. 32", "+7 435 795 58-85"));

    connect(ui->AddClient, &QPushButton::clicked, this, &ClientsMenu::RequestToAddClient);
    connect(ui->FIOEdit, &QLineEdit::textChanged, this, &ClientsMenu::InfoChanged);
    connect(ui->AdressEdit, &QLineEdit::textChanged, this, &ClientsMenu::InfoChanged);
    connect(ui->PhoneEdit, &QLineEdit::textChanged, this, &ClientsMenu::InfoChanged);
}

ClientsMenu::~ClientsMenu()
{
    auto ListBegin = ClientsList.begin();
    auto ListEnd = ClientsList.end();
    for (;ListBegin<ListEnd; ListBegin++)
    {
        delete *ListBegin;

    }
    delete ui;
}

void ClientsMenu::SetModelHeader()
{
    model->setHeaderData(0, Qt::Horizontal, "Регистрационный номер");
    model->setHeaderData(1, Qt::Horizontal, "ФИО");
    model->setHeaderData(2, Qt::Horizontal, "Адрес");
    model->setHeaderData(3, Qt::Horizontal, "Телефон");
}

void ClientsMenu::ClearInputs()
```

```
{
    ui->AddClient->setDisabled(true);

    ui->FIOEdit->clear();
    ui->AdressEdit->clear();
    ui->PhoneEdit->clear();
}

void ClientsMenu::AddNewClientToModel(Client* ClientInfo)
{
    int IndexForAdd = ++this->NumOfClients-1;
    model->setItem(IndexForAdd, 0, new QStandardItem(QString::number(ClientInfo-
>GetRegNumber())));
    model->setItem(IndexForAdd, 1, new QStandardItem(ClientInfo->GetFIO()));
    model->setItem(IndexForAdd, 2, new QStandardItem(ClientInfo->GetAdress()));
    model->setItem(IndexForAdd, 3, new QStandardItem(ClientInfo->GetPhoneNumber()));
    ClientsList.append(ClientInfo);

    ui->ClientTable->resizeRowToContents(model->rowCount() - 1);

    OrderManipulate::NewAction("Добавлен клиент " + QString::number(ClientInfo-
>GetRegNumber()) + " " + ClientInfo->GetFIO());
}

void ClientsMenu::RemoveClientFromModel(int RegNumber)
{
    auto ListBegin = ClientsList.begin();
    auto ListEnd = ClientsList.end();
    QList<Client*>::iterator finded = ListEnd;
    for (;ListBegin<ListEnd; ListBegin++)
    {
        if ((*ListBegin)->GetRegNumber() == RegNumber)
        {
            finded = ListBegin;
            break;
        }
    }

    int IndexOfFinded = finded-ClientsList.begin();
    if (finded != ListEnd)
    {
        OrderManipulate::NewAction("Удалён клиент " + QString::number((*finded)-
>GetRegNumber()) + " " + (*finded)->GetFIO());

        ClientsList.erase(finded);
        model->removeRow(IndexOfFinded);
        this->NumOfClients--;
    }


}
```

```
bool ClientsMenu::IsClientInList(int RegNumber)
{
   auto ListBegin = ClientsList.begin();
   auto ListEnd = ClientsList.end();
   for (;ListBegin<ListEnd; ListBegin++)
   {
      if ((*ListBegin)->GetRegNumber() == RegNumber)
      {
         return true;
      }
   }
   return false;
}

void ClientsMenu::RequestToAddClient()
{
   QString PhoneNumber = ui->PhoneEdit->text();
   if (not(IsClientInList(Client::PhoneNumberToRegNumber(PhoneNumber))))
   {
      AddNewClientToModel(new Client(nullptr, ui->FIOEdit->text(), ui->AdressEdit->text(),
PhoneNumber));
   }
   else
   {
      OrderManipulate::SendErrorMessage("Такой клиент уже существует.");
      return;
   }
   ClearInputs();
}

void ClientsMenu::InfoChanged()
{
   if (ui->PhoneEdit->hasAcceptableInput() && not(ui->FIOEdit->text().isEmpty()) && not(ui-
>AdressEdit->text().isEmpty()))
   {
      ui->AddClient->setDisabled(false);
   }
   else
   {
      ui->AddClient->setDisabled(true);
   }
}
```

Historymenu.cpp

```
#include "historymenu.h"
#include "ui_historymenu.h"

#include <QDate>
#include <QTime>

HistoryMenu::HistoryMenu(QWidget *parent)
   : QWidget(parent)
```

```
    , ui(new Ui::HistoryMenu)
{
    ui->setupUi(this);
    this->setWindowFlag(Qt::Window);

    auto HistoryTable = ui->HistoryTable;
    HistoryTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
    HistoryTable->horizontalHeader()->setSectionResizeMode(QHeaderView::Stretch);

    model = new QStandardItemModel(0, 3, this);

    model->setHeaderData(0, Qt::Horizontal, "Дата");
    model->setHeaderData(1, Qt::Horizontal, "Время");
    model->setHeaderData(2, Qt::Horizontal, "Действие");

    HistoryTable->setModel(model);
    HistoryTable->setWordWrap(true);
}

HistoryMenu::~HistoryMenu()
{
    delete ui;
}

void HistoryMenu::NewAction(QString InText)
{
    model->appendRow({new QStandardItem(QDate::currentDate().toString("dd.MM.yyyy")),
                      new QStandardItem(QTime::currentTime().toString("hh:mm:ss")),
                      new QStandardItem(InText)
                      });

    ui->HistoryTable->resizeRowToContents(model->rowCount() - 1);
}
```

Main.cpp

```
#include "mainwindow.h"

#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);

    MainWindow *w = new MainWindow;
    w->setAttribute(Qt::WA_DeleteOnClose);
    w->show();

    return a.exec();
}
```

Mainwindow.cpp

```
#include "mainwindow.h"
```

```
#include "ui_mainwindow.h"
#include "ordermanipulate.h"

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    centralWidget()->setCursor(Qt::ArrowCursor);

    OrderManipulate::SetRootWidget(this);

    CM = new ClientsMenu(nullptr);
    OM = new OrdersMenu(nullptr);
    MM = new MastersMenu(nullptr);
    TM = new ThingsMenu(nullptr);
    PM = new PlacesMenu(nullptr);
    HM = new HistoryMenu(nullptr);

    connect(ui->ClientsButton, &QPushButton::clicked, CM, &ClientsMenu::show);
    connect(ui->OrdersButton, &QPushButton::clicked, OM, &OrdersMenu::show);
    connect(ui->MastersButton, &QPushButton::clicked, MM, &MastersMenu::show);
    connect(ui->ThingsButton, &QPushButton::clicked, TM, &ThingsMenu::show);
    connect(ui->PlacesButton, &QPushButton::clicked, PM, &PlacesMenu::show);
    connect(ui->HistoryButton, &QPushButton::clicked, HM, &HistoryMenu::show);

    connect(OM, &OrdersMenu::RequestToAddOrder, this, &OrderManipulate::AddOrder);
    connect(OM, &OrdersMenu::RequestToExecuteOrder, this, &OrderManipulate::ExecuteOrder);
    connect(OM, &OrdersMenu::RequestToRemoveOrder, this, &OrderManipulate::RemoveOrder);

    connect(MM, &MastersMenu::RequestToRemoveMaster, this,
&OrderManipulate::RemoveMaster);

    connect(PM, &PlacesMenu::PlaceAdded, MM, &MastersMenu::UpdatePlaceList);
    connect(PM, &PlacesMenu::PlaceAdded, TM, &ThingsMenu::UpdatePlaceList);

    connect(this, &MainWindow::NewAction, HM, &HistoryMenu::NewAction);
}

MainWindow::~MainWindow()
{
    delete CM;
    delete OM;
    delete MM;
    delete TM;
    delete PM;
    delete ui;
}

ClientsMenu* MainWindow::GetClientsMenu()
{
```

```
    return CM;
}

OrdersMenu* MainWindow::GetOrdersMenu()
{
    return OM;
}

MastersMenu* MainWindow::GetMastersMenu()
{
    return MM;
}

ThingsMenu* MainWindow::GetThingsMenu()
{
    return TM;
}

PlacesMenu* MainWindow::GetPlacesMenu()
{
    return PM;
}
```

Master.cpp

```
#include "master.h"
#include "qdebug.h"

Master::Master(QObject *parent, QString FIO, QString Position, QString Place)
    : QObject{parent}
{
    this->FIO = FIO;
    this->Position = Position;
    this->Place = Place;
    this->Status = MasterStatus::InWork;
    this->MasterBusy = false;
    this->OrderNumber = -1;

    this->RegCode = Master::FIOnPosToRegCode(FIO, Position);
}

void Master::SetOrder(int OrderNumber)
{
    this->OrderNumber = OrderNumber;
    if (OrderNumber == -1)
    {
        this->MasterBusy = false;
    }
    else
    {
        this->MasterBusy = true;
    }
}
```

```
void Master::SetStatus(bool Status)
{
    if (Status)
    {
        this->Status = MasterStatus::InWork;
    }
    else
    {
        this->Status = MasterStatus::Fired;
    }
}

QString Master::GetRegCode()
{
    return this->RegCode;
}

QString Master::GetFIO()
{
    return this->FIO;
}

QString Master::GetPosition()
{
    return this->Position;
}

QString Master::GetPlaceName()
{
    return this->Place;
}

QString Master::GetStatusString()
{
    switch (this->Status)
    {
    case MasterStatus::InWork:
        return "Работает";
        break;
    case MasterStatus::Fired:
        return "Уволен";
        break;
    }
    return "";
}

Master::MasterStatus Master::GetStatus()
{
    return this->Status;
}
```

```
bool Master::IsBusy()
{
    return this->MasterBusy;
}

QString Master::GetOrderStatus()
{
    if ((this->OrderNumber == -1) && not(this->MasterBusy))
    {
        return "Нет";
    }
    return "#" + QString::number(this->OrderNumber+1);
}

QString Master::FIOnPosToRegCode(QString FIO, QString Position)
{
    QString RegCode = "";
    if (Position == "Intern")
    {
        RegCode += "IN-";
    }
    else if (Position == "Junior")
    {
        RegCode += "JN-";
    }
    else if (Position == "Middle")
    {
        RegCode += "ML-";
    }
    else if (Position == "Senior")
    {
        RegCode += "SN-";
    }
    else
    {
        RegCode += "OT-";
    }

    int tempCode = 0;
    FIO = FIO.toUpper();
    foreach (auto str, FIO.split(" ")){
        if (str.isEmpty())
        {
            continue;
        }
        tempCode += int(str[0].unicode());
    }
    return RegCode + QString::number(tempCode);
}
```

Mastersmenu.cpp

```
#include "mastersmenu.h"
```

```
#include "ordermanipulate.h"
#include "ui_mastersmenu.h"

MastersMenu::MastersMenu(QWidget *parent)
    : QWidget(parent)
    , ui(new Ui::MastersMenu)
{
    ui->setupUi(this);
    this->setWindowFlag(Qt::Window);

    model = new QStandardItemModel(0, 6, this);
    SetModelHeader();

    this->NumOfMasters = 0;
    auto MasterTable = ui->MastersTable;
    MasterTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
    MasterTable->horizontalHeader()->setSectionResizeMode(QHeaderView::Stretch);

    MasterTable->setModel(model);
    MasterTable->setWordWrap(true);
    ui->RegEdit->setInputMask(">AA-9000;_");

    AddNewMasterToModel(new Master(nullptr, "Тетерин Емельян Васильевич", "Junior", "На
ленинском"));
    AddNewMasterToModel(new Master(nullptr, "Носкова Мия Вячеславовна", "Junior", "На
московской"));
    AddNewMasterToModel(new Master(nullptr, "Карпов Иван Филатович", "Senior", "На
ленинском"));
    AddNewMasterToModel(new Master(nullptr, "Ефимов Людвиг Анатольевич", "Middle", "На
московской"));

    connect(ui->AddMaster, &QPushButton::clicked, this, &MastersMenu::RequestToAddMaster);
    connect(ui->RemoveMaster, &QPushButton::clicked, this, [this] {emit
RequestToRemoveMaster(ui->RegEdit->text());});
    connect(ui->PositionCombo, &QComboBox::currentTextChanged, this,
&MastersMenu::InfoChanged);
    connect(ui->FIOEdit, &QLineEdit::textChanged, this, &MastersMenu::InfoChanged);
    connect(ui->RegEdit, &QLineEdit::textChanged, this, &MastersMenu::RegNumberChanged);
}

MastersMenu::~MastersMenu()
{
    auto ListBegin = MasterList.begin();
    auto ListEnd = MasterList.end();
    for (;ListBegin<ListEnd; ListBegin++)
    {
        delete *ListBegin;

    }
    delete ui;
}
```

```
void MastersMenu::SetMasterBusyStatusByRegCode(QString RegCode, int OrderNumber)
{
   auto ListBegin = MasterList.begin();
   auto ListEnd = MasterList.end();
   for (;ListBegin<ListEnd; ListBegin++)
   {
      if ((*ListBegin)->GetRegCode() == RegCode)
      {
         break;
      }
   }

   (*ListBegin)->SetOrder(OrderNumber);
   model->setItem(ListBegin-MasterList.begin(), 5, new QStandardItem((*ListBegin)-
>GetOrderStatus()));
}

void MastersMenu::SetModelHeader()
{
   model->setHeaderData(0, Qt::Horizontal, "Номер");
   model->setHeaderData(1, Qt::Horizontal, "ФИО");
   model->setHeaderData(2, Qt::Horizontal, "Должность");
   model->setHeaderData(3, Qt::Horizontal, "Точка");
   model->setHeaderData(4, Qt::Horizontal, "Статус");
   model->setHeaderData(5, Qt::Horizontal, "Заказ");
}

void MastersMenu::ClearInputs()
{
   ui->AddMaster->setDisabled(true);
   ui->RemoveMaster->setDisabled(true);

   ui->FIOEdit->clear();
   ui->PositionCombo->setCurrentIndex(0);
   ui->RegEdit->clear();
}

void MastersMenu::AddNewMasterToModel(Master* MasterInfo)
{
   int IndexForAdd = ++this->NumOfMasters-1;
   model->setItem(IndexForAdd, 0, new QStandardItem(MasterInfo->GetRegCode()));
   model->setItem(IndexForAdd, 1, new QStandardItem(MasterInfo->GetFIO()));
   model->setItem(IndexForAdd, 2, new QStandardItem(MasterInfo->GetPosition()));
   model->setItem(IndexForAdd, 3, new QStandardItem(MasterInfo->GetPlaceName()));
   model->setItem(IndexForAdd, 4, new QStandardItem(MasterInfo->GetStatusString()));
   model->setItem(IndexForAdd, 5, new QStandardItem(MasterInfo->GetOrderStatus()));
   MasterList.append(MasterInfo);

   ui->MastersTable->resizeRowToContents(model->rowCount() - 1);

   OrderManipulate::NewAction("Добавлен мастер " + MasterInfo->GetRegCode() + " " +
MasterInfo->GetFIO());
```

```
}

void MastersMenu::UpdatePlaceList(QString PlaceName)
{
   ui->PlaceComboBox->addItem(PlaceName);
}

void MastersMenu::FireMaster(QString RegCode)
{
   auto ListBegin = MasterList.begin();
   auto ListEnd = MasterList.end();
   QList<Master*>::iterator finded = ListEnd;
   for (;ListBegin<ListEnd; ListBegin++)
   {
      if ((*ListBegin)->GetRegCode() == RegCode)
      {
         finded = ListBegin;
         break;
      }
   }
   int IndexOfFinded = finded-MasterList.begin();
   if (finded != ListEnd)
   {
      OrderManipulate::NewAction("Мастер " + (*finded)->GetRegCode() + " " + (*finded)-
>GetFIO() + " уволен");
      (*finded)->SetStatus(false);
      model->setItem(IndexOfFinded, 4, new QStandardItem((*finded)->GetStatusString()));
   }
}

bool MastersMenu::IsMasterInList(QString RegCode)
{
   auto ListBegin = MasterList.begin();
   auto ListEnd = MasterList.end();
   for (;ListBegin<ListEnd; ListBegin++)
   {
      if ((*ListBegin)->GetRegCode() == RegCode)
      {
         return true;
      }
   }
   return false;
}

Master* MastersMenu::GetMasterByRegCode(QString RegCode)
{
   auto ListBegin = MasterList.begin();
   auto ListEnd = MasterList.end();
   for (;ListBegin<ListEnd; ListBegin++)
   {
      if ((*ListBegin)->GetRegCode() == RegCode)
      {
```

```
            return *ListBegin;
        }
    }
    return nullptr;
}

void MastersMenu::RequestToAddMaster()
{
    QString FIO = ui->FIOEdit->text();
    QString Position = ui->PositionCombo->currentText();
    QString RegCode = Master::FIOnPosToRegCode(FIO, Position);
    QString PlaceName = ui->PlaceComboBox->currentText();
    if (not(IsMasterInList(RegCode)))
    {
        AddNewMasterToModel(new Master(nullptr, FIO, Position, PlaceName));
    }
    else
    {
        OrderManipulate::SendErrorMessage("Такой мастер уже существует.");
        return;
    }
    ClearInputs();
}

void MastersMenu::InfoChanged()
{
    if (not(ui->PositionCombo->currentText().isEmpty()) && not(ui->FIOEdit->text().isEmpty())
&&
        not(ui->FIOEdit->text().size() > 31))
    {
        ui->AddMaster->setDisabled(false);
    }
    else
    {
        ui->AddMaster->setDisabled(true);
    }
}

void MastersMenu::RegNumberChanged()
{
    if (ui->RegEdit->hasAcceptableInput() && NumOfMasters>0)
    {
        ui->RemoveMaster->setDisabled(false);
    }
    else
    {
        ui->RemoveMaster->setDisabled(true);
    }
}
```

Order.cpp

```
#include "order.h"
```

```
Order::Order(QObject *parent, int ClientPtr,
            QString MasterPtr, QString ThingToRepair)
   : QObject{parent}
{
   this->ClientRegNumber = ClientPtr;
   this->MasterRegCode = MasterPtr;
   this->ThingToRepair = ThingToRepair;
   this->StatusOfRepair = RepairStatus::InProcess;
}

void Order::SetRepairStatus(RepairStatus Status)
{
   this->StatusOfRepair = Status;
}

int Order::GetClientRegNumber()
{
   return ClientRegNumber;
}

QString Order::GetMasterRegCode()
{
   return MasterRegCode;
}

QString Order::GetThingToRepair()
{
   return this->ThingToRepair;
}

Order::RepairStatus Order::GetStatusOfRepair()
{
   return StatusOfRepair;
}

QString Order::GetStringStatusOfRepair()
{
   switch (this->StatusOfRepair)
   {
   case RepairStatus::InProcess:
      return "В процессе";
      break;
   case RepairStatus::Cancelled:
      return "Отменён";
      break;
   case RepairStatus::Completed:
      return "Выполнен";
      break;
   }
   return "";
}
```

Ordermanupulate.cpp

```
#include "ordermanipulate.h"
#include "qmessagebox.h"

MainWindow* OrderManipulate::RootWidget = nullptr;

OrderManipulate::OrderManipulate(QWidget* parent)
   : QWidget{parent}
{}

void OrderManipulate::SetRootWidget(MainWindow* InRootWidget)
{
   RootWidget = InRootWidget;
}

void OrderManipulate::SendErrorMessage(QString ErrorMessage)
{
   QMessageBox msgBox;
   msgBox.setText(ErrorMessage);
   msgBox.setIcon(QMessageBox::Critical);
   msgBox.exec();
}

void OrderManipulate::AddOrder(Order* OrderInfo)
{
   QString MasterRegCode = OrderInfo->GetMasterRegCode();
   QString ThingName = OrderInfo->GetThingToRepair();
   if (not(RootWidget->GetClientsMenu()->IsClientInList(OrderInfo->GetClientRegNumber())))
   {
      OrderManipulate::SendErrorMessage("Такого клиента не существует.");

      delete OrderInfo;
      return;
   }
   if (not(RootWidget->GetMastersMenu()->IsMasterInList(MasterRegCode)))
   {
      OrderManipulate::SendErrorMessage("Такого мастера не существует.");

      delete OrderInfo;
      return;
   }
   if (not(RootWidget->GetThingsMenu()->IsThingNameInList(ThingName)))
   {
      OrderManipulate::SendErrorMessage("Данная вещь не зарегистрирована.");

      delete OrderInfo;
      return;
   }
   Thing* ThingInfo = RootWidget->GetThingsMenu()->GetThingByName(ThingName);
   if (RootWidget->GetPlacesMenu()->GetPlaceByName(ThingInfo->GetPlaceName())-
>GetPlaceStatus() == Place::PlaceStatus::Suspended)
```

```
    {
        OrderManipulate::SendErrorMessage("Данная точка не принимает заказы.");

        delete OrderInfo;
        return;
    }
    if (ThingInfo->IsThingInOrder())
    {
        OrderManipulate::SendErrorMessage("Данная вещь уже в работе.");

        delete OrderInfo;
        return;
    }
    Master* MasterInfo = RootWidget->GetMastersMenu()-
>GetMasterByRegCode(MasterRegCode);
    if (MasterInfo->GetStatus() == Master::MasterStatus::Fired)
    {
        OrderManipulate::SendErrorMessage("Данный мастер уже уволен.");

        return;
    }
    if (MasterInfo->IsBusy())
    {
        OrderManipulate::SendErrorMessage("Мастер уже занят.");

        delete OrderInfo;
        return;
    }
    if (ThingInfo->GetPlaceName() != MasterInfo->GetPlaceName())
    {
        OrderManipulate::SendErrorMessage("Места мастера и вещи не совпадают.");

        delete OrderInfo;
        return;
    }

    int OrderNumber = RootWidget->GetOrdersMenu()->GetSizeOfList();

    RootWidget->GetOrdersMenu()->AddNewOrderToModel(OrderInfo);
    RootWidget->GetMastersMenu()->SetMasterBusyStatusByRegCode(MasterRegCode,
OrderNumber);
    RootWidget->GetThingsMenu()->SetOrderStatusByName(ThingName, OrderNumber);
    RootWidget->GetOrdersMenu()->ClearInputs();
}

void OrderManipulate::ExecuteOrder(int OrderNumber)
{

    if (OrderNumber > RootWidget->GetOrdersMenu()->GetSizeOfList())
    {
        OrderManipulate::SendErrorMessage("Такого заказа не существует.");
```

```
        return;
    }
    Order* OrderInfo = RootWidget->GetOrdersMenu()->GetOrderByNumber(OrderNumber);
    if (OrderInfo->GetStatusOfRepair() != Order::RepairStatus::InProcess)
    {
        OrderManipulate::SendErrorMessage("Данный заказ уже закончен.");

        return;
    }

    RootWidget->GetMastersMenu()->SetMasterBusyStatusByRegCode(OrderInfo-
>GetMasterRegCode(), -1);
    RootWidget->GetOrdersMenu()->SetOrderAsCompleted(OrderNumber);
    RootWidget->GetThingsMenu()->SetOrderStatusByName(OrderInfo->GetThingToRepair(), -1);
    RootWidget->GetOrdersMenu()->ClearInputs();
}

void OrderManipulate::RemoveOrder(int OrderNumber)
{
    if (OrderNumber > RootWidget->GetOrdersMenu()->GetSizeOfList())
    {
        OrderManipulate::SendErrorMessage("Такого заказа не существует.");

        return;
    }
    Order* OrderInfo = RootWidget->GetOrdersMenu()->GetOrderByNumber(OrderNumber);
    if (OrderInfo->GetStatusOfRepair() != Order::RepairStatus::InProcess)
    {
        OrderManipulate::SendErrorMessage("Данный заказ уже закончен.");

        return;
    }
    RootWidget->GetOrdersMenu()->CancelOrder(OrderNumber);
    RootWidget->GetMastersMenu()->SetMasterBusyStatusByRegCode(OrderInfo-
>GetMasterRegCode(), -1);
    RootWidget->GetThingsMenu()->SetOrderStatusByName(OrderInfo->GetThingToRepair(), -1);
    RootWidget->GetOrdersMenu()->ClearInputs();
}

void OrderManipulate::RemoveMaster(QString RegCode)
{
    if (not(RootWidget->GetMastersMenu()->IsMasterInList(RegCode)))
    {
        OrderManipulate::SendErrorMessage("Такого мастера не существует.");

        return;
    }
    Master* MasterInfo = RootWidget->GetMastersMenu()->GetMasterByRegCode(RegCode);
    if (MasterInfo->IsBusy())
    {
        OrderManipulate::SendErrorMessage("У мастера есть не выполненный заказ.");
```

```
        return;
    }
    if (MasterInfo->GetStatus() == Master::MasterStatus::Fired)
    {
        OrderManipulate::SendErrorMessage("Данный мастер уже уволен.");

        return;
    }
    RootWidget->GetMastersMenu()->FireMaster(RegCode);
    RootWidget->GetMastersMenu()->ClearInputs();
}

void OrderManipulate::NewAction(QString InMessage)
{
    RootWidget->NewAction(InMessage);
}
```

Ordersmenu.cpp

```
#include "ordersmenu.h"
#include "ui_ordersmenu.h"
#include "ordermanipulate.h"

OrdersMenu::OrdersMenu(QWidget *parent)
    : QWidget(parent)
    , ui(new Ui::OrdersMenu)
{
    ui->setupUi(this);
    this->setWindowFlag(Qt::Window);

    this->NumOfOrders = 0;
    auto OrdersTable = ui->OrdersTable;
    OrdersTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
    OrdersTable->horizontalHeader()->setSectionResizeMode(QHeaderView::Stretch);

    model = new QStandardItemModel(0, 4, this);
    SetModelHeader();
    OrdersTable->setModel(model);
    OrdersTable->setWordWrap(true);

    ui->RegNumClientEdit->setValidator(new QIntValidator(0, 810, this));
    ui->RegNumMasterEdit->setInputMask(">AA-9000;_");
    auto Validator = new QIntValidator(this);
    Validator->setBottom(1);
    ui->NumOfRowEdit->setValidator(Validator);

    connect(ui->RegNumClientEdit, &QLineEdit::textChanged, this, &OrdersMenu::InfoChanged);
    connect(ui->RegNumMasterEdit, &QLineEdit::textChanged, this, &OrdersMenu::InfoChanged);
    connect(ui->ThingEdit, &QLineEdit::textChanged, this, &OrdersMenu::InfoChanged);
    connect(ui->NumOfRowEdit, &QLineEdit::textChanged, this,
&OrdersMenu::RowNumberChanged);

    connect(ui->AddOrder, &QPushButton::clicked, this, [this] {
```

```
    emit RequestToAddOrder(new Order(nullptr, ui->RegNumClientEdit->text().toInt(), ui->RegNumMasterEdit->text(), ui->ThingEdit->text()));});
    connect(ui->ExecuteOrder, &QPushButton::clicked, this, [this] {emit RequestToExecuteOrder(ui->NumOfRowEdit->text().toInt());});
    connect(ui->RemoveOrder, &QPushButton::clicked, this, [this] {emit RequestToRemoveOrder(ui->NumOfRowEdit->text().toInt());});
}

OrdersMenu::~OrdersMenu()
{
    auto ListBegin = OrderList.begin();
    auto ListEnd = OrderList.end();
    for (;ListBegin<ListEnd; ListBegin++)
    {
        delete *ListBegin;

    }
    delete ui;
}

int OrdersMenu::GetSizeOfList()
{
    return this->NumOfOrders;
}

void OrdersMenu::SetModelHeader()
{
    model->setHeaderData(0, Qt::Horizontal, "Номер клиента");
    model->setHeaderData(1, Qt::Horizontal, "Номер мастера");
    model->setHeaderData(2, Qt::Horizontal, "Объект заказа");
    model->setHeaderData(3, Qt::Horizontal, "Статус");
}

void OrdersMenu::ClearInputs()
{
    ui->AddOrder->setDisabled(true);
    ui->RemoveOrder->setDisabled(true);

    ui->RegNumClientEdit->clear();
    ui->RegNumMasterEdit->clear();
    ui->ThingEdit->clear();
    ui->NumOfRowEdit->clear();
}

void OrdersMenu::AddNewOrderToModel(Order* OrderInfo)
{
    int IndexForAdd = ++this->NumOfOrders-1;
    model->setItem(IndexForAdd, 0, new QStandardItem(QString::number(OrderInfo->GetClientRegNumber())));
    model->setItem(IndexForAdd, 1, new QStandardItem(OrderInfo->GetMasterRegCode()));
    model->setItem(IndexForAdd, 2, new QStandardItem(OrderInfo->GetThingToRepair()));
    model->setItem(IndexForAdd, 3, new QStandardItem(OrderInfo->GetStringStatusOfRepair()));
```

```
    OrderList.append(OrderInfo);

    ui->OrdersTable->resizeRowToContents(model->rowCount() - 1);

    OrderManipulate::NewAction("Добавлен заказ клиента " + QString::number(OrderInfo-
>GetClientRegNumber()) + " мастеру " + OrderInfo->GetMasterRegCode());
}

void OrdersMenu::CancelOrder(int OrderNumber)
{
    Order* OrderInfo = GetOrderByNumber(OrderNumber);
    OrderInfo->SetRepairStatus(Order::RepairStatus::Cancelled);
    model->setItem(OrderNumber-1, 3, new QStandardItem(OrderInfo-
>GetStringStatusOfRepair()));
    OrderManipulate::NewAction("Отменён заказ клиента " + QString::number(OrderInfo-
>GetClientRegNumber()) + " мастеру " + OrderInfo->GetMasterRegCode());
}

Order *OrdersMenu::GetOrderByNumber(int OrderNumber)
{
    return OrderList.at(OrderNumber-1);
}

void OrdersMenu::SetOrderAsCompleted(int OrderNumber)
{
    auto OrderInfo = GetOrderByNumber(OrderNumber);
    OrderInfo->SetRepairStatus(Order::RepairStatus::Completed);
    model->setItem(OrderNumber-1, 3, new QStandardItem(OrderInfo-
>GetStringStatusOfRepair()));
    OrderManipulate::NewAction("Выполнен заказ клиента " + QString::number(OrderInfo-
>GetClientRegNumber()) + " мастеру " + OrderInfo->GetMasterRegCode());
}

void OrdersMenu::InfoChanged()
{
    if (ui->RegNumClientEdit->hasAcceptableInput() && ui->RegNumMasterEdit-
>hasAcceptableInput() &&
        not(ui->ThingEdit->text().isEmpty()))
    {
        ui->AddOrder->setDisabled(false);
    }
    else
    {
        ui->AddOrder->setDisabled(true);
    }
}

void OrdersMenu::RowNumberChanged()
{
    if (ui->NumOfRowEdit->hasAcceptableInput() && NumOfOrders>0)
    {
        ui->RemoveOrder->setDisabled(false);
```

```
        ui->ExecuteOrder->setDisabled(false);
    }
    else
    {
        ui->RemoveOrder->setDisabled(true);
        ui->ExecuteOrder->setDisabled(true);
    }
}
```

Place.cpp

```
#include "place.h"


Place::Place(QObject *parent, QString PlaceName, QString PlaceAdress, QString PhoneNumber,
PlaceStatus Status): QObject{parent}
{
    this->PlaceName=PlaceName;
    this->PlaceAdress = PlaceAdress;
    this->PhoneNumber=PhoneNumber;
    this->Status=Status;
}

QString Place::GetPlaceName()
{
    return PlaceName;
}

QString Place::GetPlaceAdress()
{
    return PlaceAdress;
}

QString Place::GetPhoneNumber()
{
    return PhoneNumber;
}

Place::PlaceStatus Place::GetPlaceStatus()
{
    return Status;
}

QString Place::GetStringPlaceStatus()
{
    switch (this->Status)
    {
    case PlaceStatus::InWork:
        return "Работает";
        break;
    case PlaceStatus::Suspended:
        return "Приостановлен";
        break;
```

```
    }
    return "";
}

void Place::EnablePlace()
{
    this->Status = PlaceStatus::InWork;
}

void Place::DisablePlace()
{
    this->Status = PlaceStatus::Suspended;
}
```

Placesmenu.cpp

```
#include "placesmenu.h"
#include "ui_placesmenu.h"
#include "ordermanipulate.h"
#include <QTimer>

PlacesMenu::PlacesMenu(QWidget *parent)
    : QWidget(parent)
    , ui(new Ui::PlacesMenu)
{
    ui->setupUi(this);
    this->setWindowFlag(Qt::Window);

    this->NumOfPlaces = 0;
    auto PlacesTable = ui->PlacesTable;
    PlacesTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
    PlacesTable->horizontalHeader()->setSectionResizeMode(QHeaderView::Stretch);

    model = new QStandardItemModel(0, 4, this);
    SetModelHeader();
    PlacesTable->setModel(model);
    PlacesTable->setWordWrap(true);

    ui->PhoneNumEdit->setInputMask("+7 999 999 99-99;_");
    auto Validator = new QIntValidator(this);
    Validator->setBottom(1);
    ui->NumOfRowEdit_1->setValidator(Validator);

    AddNewPlaceToModel(new Place(nullptr, "На московской", "Пр.Московский", "+7 94324
4342 234"));
    AddNewPlaceToModel(new Place(nullptr, "На ленинском", "Пр.Ленинский", "+7 94324 4542
234"));
    AddNewPlaceToModel(new Place(nullptr, "На петроградской", "Пр. Кронверский", "+7 924
4342 234"));

    connect(ui->AddPlace, &QPushButton::clicked, this, &PlacesMenu::RequestToAddPlace);
    connect(ui->EnablePlace, &QPushButton::clicked, this, &PlacesMenu::EnablePlace);
    connect(ui->StopPlace, &QPushButton::clicked, this, &PlacesMenu::StopPlace);
```

```
    connect(ui->NameOfPlaceEdit, &QLineEdit::textChanged, this, &PlacesMenu::InfoChanged);
    connect(ui->AdressOfPlaceEdit, &QLineEdit::textChanged, this, &PlacesMenu::InfoChanged);
    connect(ui->PhoneNumEdit, &QLineEdit::textChanged, this, &PlacesMenu::InfoChanged);
    connect(ui->NumOfRowEdit_1, &QLineEdit::textChanged, this,
&PlacesMenu::RowNumberChanged);
}

PlacesMenu::~PlacesMenu()
{
    auto ListBegin = PlacesList.begin();
    auto ListEnd = PlacesList.end();
    for (;ListBegin<ListEnd; ListBegin++)
    {
        delete *ListBegin;
    }
    delete ui;
}

void PlacesMenu::ClearInputs()
{
    ui->AddPlace->setDisabled(true);
    ui->EnablePlace->setDisabled(true);
    ui->StopPlace->setDisabled(true);

    ui->NameOfPlaceEdit->clear();
    ui->AdressOfPlaceEdit->clear();
    ui->PhoneNumEdit->clear();
    ui->NumOfRowEdit_1->clear();
}

Place *PlacesMenu::GetPlaceByNumber(int PlaceNumber)
{
    return PlacesList.at(PlaceNumber-1);
}

Place *PlacesMenu::GetPlaceByName(QString PlaceName)
{
    auto ListBegin = PlacesList.begin();
    auto ListEnd = PlacesList.end();
    for (;ListBegin<ListEnd; ListBegin++)
    {
        if ((*ListBegin)->GetPlaceName() == PlaceName)
        {
            return *ListBegin;
        }
    }
    return nullptr;
}

void PlacesMenu::SetPlaceStatusByName(QString PlaceName, bool Status)
{
```

```
    auto ListBegin = PlacesList.begin();
    auto ListEnd = PlacesList.end();
    for (;ListBegin<ListEnd; ListBegin++)
    {
        if ((*ListBegin)->GetPlaceName() == PlaceName)
        {
            break;
        }
    }

    if (Status)
    {
        (*ListBegin)->EnablePlace();
    }
    else
    {
        (*ListBegin)->DisablePlace();
    }
}

void PlacesMenu::SetModelHeader()
{
    model->setHeaderData(0, Qt::Horizontal, "Наименование");
    model->setHeaderData(1, Qt::Horizontal, "Адрес");
    model->setHeaderData(2, Qt::Horizontal, "Номер телефона");
    model->setHeaderData(3, Qt::Horizontal, "Статус");
}

void PlacesMenu::AddNewPlaceToModel(Place* PlaceInfo)
{
    int IndexForAdd = ++this->NumOfPlaces-1;
    model->setItem(IndexForAdd, 0, new QStandardItem(PlaceInfo->GetPlaceName()));
    model->setItem(IndexForAdd, 1, new QStandardItem(PlaceInfo->GetPlaceAdress()));
    model->setItem(IndexForAdd, 2, new QStandardItem(PlaceInfo->GetPhoneNumber()));
    model->setItem(IndexForAdd, 3, new QStandardItem(PlaceInfo->GetStringPlaceStatus()));
    PlacesList.append(PlaceInfo);

    ui->PlacesTable->resizeRowToContents(model->rowCount() - 1);

    QTimer::singleShot(0, this, [PlaceInfo, this]() {
        emit PlaceAdded(PlaceInfo->GetPlaceName());
        OrderManipulate::NewAction("Добавлена новая точка " + PlaceInfo->GetPlaceName() + "
по адресу " + PlaceInfo->GetPlaceAdress());
    });

}

void PlacesMenu::RequestToAddPlace()
{
    QString PlaceName = ui->NameOfPlaceEdit->text();
    QString PlaceNumber = ui->PhoneNumEdit->text();
    QString PlaceAdress = ui->AdressOfPlaceEdit->text();
```

```
    if (IsPlaceNameInList(PlaceName))
    {
        OrderManipulate::SendErrorMessage("Такое название точки уже существует.");
        return;
    }
    else if (IsPlaceAdressInList(PlaceAdress))
    {
        OrderManipulate::SendErrorMessage("Такой адрес точки уже зарегистрирован.");
        return;
    }
    else
    {
        AddNewPlaceToModel(new Place(nullptr, PlaceName, PlaceAdress, PlaceNumber));
    }
    ClearInputs();
}

bool PlacesMenu::IsPlaceNameInList(QString PlaceName)
{
    auto ListBegin = PlacesList.begin();
    auto ListEnd = PlacesList.end();
    for (;ListBegin<ListEnd; ListBegin++)
    {
        if ((*ListBegin)->GetPlaceName() == PlaceName)
        {
            return true;
        }
    }
    return false;
}

bool PlacesMenu::IsPlaceAdressInList(QString PlaceAdress)
{
    auto ListBegin = PlacesList.begin();
    auto ListEnd = PlacesList.end();
    for (;ListBegin<ListEnd; ListBegin++)
    {
        if ((*ListBegin)->GetPlaceAdress() == PlaceAdress)
        {
            return true;
        }
    }
    return false;
}

void PlacesMenu::EnablePlace()
{
    int PlaceNumber = ui->NumOfRowEdit_1->text().toInt();
    if (PlaceNumber > NumOfPlaces)
    {
        OrderManipulate::SendErrorMessage("Такой точки не существует.");
```

```
        return;
    }
    Place* PlaceInfo = GetPlaceByNumber(PlaceNumber);
    PlaceInfo->EnablePlace();
    model->setItem(PlaceNumber-1, 3, new QStandardItem(PlaceInfo->GetStringPlaceStatus()));
    ClearInputs();

    OrderManipulate::NewAction("Точка " + PlaceInfo->GetPlaceName() + " по адресу " +
PlaceInfo->GetPlaceAdress() + " теперь принимает заказы");
}

void PlacesMenu::StopPlace()
{
    int PlaceNumber = ui->NumOfRowEdit_1->text().toInt();
    if (PlaceNumber > NumOfPlaces)
    {
        OrderManipulate::SendErrorMessage("Такой точки не существует.");

        return;
    }
    Place* PlaceInfo = GetPlaceByNumber(PlaceNumber);
    PlaceInfo->DisablePlace();
    model->setItem(PlaceNumber-1, 3, new QStandardItem(PlaceInfo->GetStringPlaceStatus()));
    ClearInputs();

    OrderManipulate::NewAction("Точка " + PlaceInfo->GetPlaceName() + " по адресу " +
PlaceInfo->GetPlaceAdress() + " теперь не принимает заказы");
}

void PlacesMenu::InfoChanged()
{
    if (ui->PhoneNumEdit->hasAcceptableInput() && not(ui->NameOfPlaceEdit->text().isEmpty())
&& not(ui->AdressOfPlaceEdit->text().isEmpty()))
    {
        ui->AddPlace->setDisabled(false);
    }
    else
    {
        ui->AddPlace->setDisabled(true);
    }
}

void PlacesMenu::RowNumberChanged()
{
    if (ui->NumOfRowEdit_1->hasAcceptableInput() && NumOfPlaces>0)
    {
        ui->EnablePlace->setDisabled(false);
        ui->StopPlace->setDisabled(false);
    }
    else
    {
        ui->EnablePlace->setDisabled(true);
```

```
        ui->StopPlace->setDisabled(true);
    }
}
```

Thing.cpp

```
#include "thing.h"

Thing::Thing(QObject *parent, QString ThingName, QString PlaceName): QObject{parent}
{
    this->ThingName = ThingName;
    this->PlaceName = PlaceName;
    this->Status = ThingStatus::None;
    this->ThingInOrder = false;
    this->OrderNumber = -1;
}

QString Thing::GetThingName()
{
    return ThingName;
}

QString Thing::GetPlaceName()
{
    return PlaceName;
}

QString Thing::GetThingStatusString()
{
    switch (this->Status)
    {
    case ThingStatus::InWork:
        return "В ремонте";
        break;
    case ThingStatus::None:
        return "Отсутствует";
        break;
    }
    return "";
}

bool Thing::IsThingInOrder()
{
    switch (this->Status)
    {
    case ThingStatus::InWork:
        return true;
        break;
    case ThingStatus::None:
        return false;
        break;
    }
    return false;
```

```
}

void Thing::SetOrder(int OrderNumber)
{
   this->OrderNumber = OrderNumber;
   switch (this->OrderNumber)
   {
   case -1:
      this->Status = ThingStatus::None;
      break;
   default:
      this->Status = ThingStatus::InWork;
      break;
   }
}
```

Thingsmenu.cpp

```
#include "thingsmenu.h"
#include "ui_thingsmenu.h"
#include "ordermanipulate.h"

ThingsMenu::ThingsMenu(QWidget *parent)
   : QWidget(parent)
   , ui(new Ui::ThingsMenu)
{
   ui->setupUi(this);
   this->setWindowFlag(Qt::Window);

   this->NumOfThings = 0;
   auto ThingsTable = ui->ThingsTable;
   ThingsTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
   ThingsTable->horizontalHeader()->setSectionResizeMode(QHeaderView::Stretch);

   model = new QStandardItemModel(0, 3, this);
   SetModelHeader();
   ThingsTable->setModel(model);
   ThingsTable->setWordWrap(true);

   connect(ui->AddThing, &QPushButton::clicked, this, &ThingsMenu::RequestToAddThing);
   connect(ui->ThingNameEdit, &QLineEdit::textChanged, this, &ThingsMenu::InfoChanged);
}

ThingsMenu::~ThingsMenu()
{
   auto ListBegin = ThingsList.begin();
   auto ListEnd = ThingsList.end();
   for (;ListBegin<ListEnd; ListBegin++)
   {
      delete *ListBegin;
   }
   delete ui;
}
```

```
void ThingsMenu::ClearInputs()
{
   ui->AddThing->setDisabled(true);

   ui->ThingNameEdit->clear();
   ui->PlaceComboBox->setCurrentIndex(0);
}

void ThingsMenu::SetOrderStatusByName(QString ThingName, int OrderNumber)
{
   auto ListBegin = ThingsList.begin();
   auto ListEnd = ThingsList.end();
   for (;ListBegin<ListEnd; ListBegin++)
   {
      if ((*ListBegin)->GetThingName() == ThingName)
      {
         break;
      }
   }

   (*ListBegin)->SetOrder(OrderNumber);
   model->setItem(ListBegin-ThingsList.begin(), 2, new QStandardItem((*ListBegin)-
>GetThingStatusString()));
}

bool ThingsMenu::IsThingNameInList(QString ThingName)
{
   auto ListBegin = ThingsList.begin();
   auto ListEnd = ThingsList.end();
   for (;ListBegin<ListEnd; ListBegin++)
   {
      if ((*ListBegin)->GetThingName() == ThingName)
      {
         return true;
      }
   }
   return false;
}

Thing* ThingsMenu::GetThingByName(QString ThingName)
{
   auto ListBegin = ThingsList.begin();
   auto ListEnd = ThingsList.end();
   for (;ListBegin<ListEnd; ListBegin++)
   {
      if ((*ListBegin)->GetThingName() == ThingName)
      {
         return *ListBegin;
      }
   }
   return nullptr;
```

```
}

void ThingsMenu::AddNewThingToModel(Thing* ThingInfo)
{
   int IndexForAdd = ++this->NumOfThings-1;
   model->setItem(IndexForAdd, 0, new QStandardItem(ThingInfo->GetThingName()));
   model->setItem(IndexForAdd, 1, new QStandardItem(ThingInfo->GetPlaceName()));
   model->setItem(IndexForAdd, 2, new QStandardItem(ThingInfo->GetThingStatusString()));
   ThingsList.append(ThingInfo);

   ui->ThingsTable->resizeRowToContents(model->rowCount() - 1);

   OrderManipulate::NewAction("Вещь " + ThingInfo->GetThingName() + " зарегистрирована
на точку " + ThingInfo->GetPlaceName());
}

void ThingsMenu::SetModelHeader()
{
   model->setHeaderData(0, Qt::Horizontal, "Наименование");
   model->setHeaderData(1, Qt::Horizontal, "Точка ремонта");
   model->setHeaderData(2, Qt::Horizontal, "Статус");
}

void ThingsMenu::RequestToAddThing()
{
   QString ThingName = ui->ThingNameEdit->text();
   QString PlaceName = ui->PlaceComboBox->currentText();
   if (IsThingNameInList(ThingName))
   {
      OrderManipulate::SendErrorMessage("Такое название вещи уже существует.");
      return;
   }
   else
   {
      AddNewThingToModel(new Thing(nullptr, ThingName, PlaceName));
   }
   ClearInputs();
}

void ThingsMenu::UpdatePlaceList(QString PlaceName)
{
   ui->PlaceComboBox->addItem(PlaceName);
}

void ThingsMenu::InfoChanged()
{
   if (not(ui->ThingNameEdit->text().isEmpty()))
   {
      ui->AddThing->setDisabled(false);
   }
   else
   {
```

```
        ui->AddThing->setDisabled(true);
    }
}
```